

Chapter 76B · Business Analyst Question Bank

Expanded to 95 questions, including the new ambiguity drill. 3 October 2026.

Chapter 76B. Business Analyst Question Bank

Part 8 — Be Interview Ready

> **Chapter at a glance** > > **You will learn to:** answer the requirements-gathering, process-mapping, and documentation questions that come up in BA interviews · tell a BRD, FRD, and SRS apart without hesitating · write a user story with real acceptance criteria, not a vague wish · map a process the way a BA actually would, with decision points and swimlanes · handle "can you just change the number?" and other stakeholder pushback without folding or getting defensive · prioritise a backlog with the named frameworks and say honestly what each one is bad at · hold your own on story points, velocity, refinement and MVP without reciting Agile slogans · **take a deliberately vague request and produce the questions that must be answered before anything is built** · talk about Jira, wireframes, data dictionaries and estimation in terms of what you produced in them · walk into an unfamiliar domain — logistics, e-commerce, financial services — and find the few things that decide the answer. > > **Before you start:** Chapter 24 (all of it: turning an ask into a question, business rules, stakeholders, and pushback) and Chapter 25 (all of it: the business analyst track), which between them teach every technique this bank tests; Chapter 26, section 26.10 (Agile, Scrum, and the backlog); Chapter 3, section 3.2 (order 5001's journey from enquiry to cash, which the worked examples use); and Chapter 69 for the three answer tiers and the twelve extra-point tags. This chapter tests those skills; it doesn't teach them again. When you can't answer a question, its **Learn it in** line sends you to the section that teaches it. > > **Time needed:** 6–7 hours to read and drill once (about 10 minutes per core question answered aloud, 1–2 minutes per rapid-fire row), plus 4–6 hours for the project: two interviews, a swimlane map, a story, and a requirement. **Section 76B.11, the ambiguity drill, is worth its own sitting of about an hour**, and it is the one to do out loud rather than read. > > **How this chapter is built.** Same format as every question bank in Part 8. Every **core question** gives a memory hook ("Remember it as..."), a one-line answer you can recall under pressure, and a tier table: what **passes**, what's **strong**, and the **extra points** (Chapter 69's moves, tagged the same way: **[+Clarify]**, **[+Edge cases]**, **[+Validate]** and so on). Then come the likely follow-ups, the red flag, and where to learn it. Every **rapid-fire section** is a scan table: question, one-line answer, one extra point, level, and where to learn it (a bare number such as 25.8 means that section). This chapter has no code to execute; a requirements document, a process map, and a user story are judged on completeness and clarity, so every worked example here is a full, concrete artifact, not a description of one. The worked examples use Riverstone's order-to-cash process, the same one Chapter 25 maps, with the same steps, requirement numbers, and figures. > > **Two sections break the format deliberately.** Section 76B.9 marks the named prioritisation frameworks — MoSCoW, RICE, Kano, WSJF — as **Beyond the book**, because Chapter 25, section 25.12 teaches this book's own ranking method and not those four; each carries a one-line primer before the questions that use it. Section 76B.11, **the ambiguity drill**, asks you to produce the *questions* rather than an answer, so instead of a worked

solution each one gives the questions that matter, says which of them move the number most, and ends with the reply you would actually send. > > **Where this chapter quotes a figure, it is measured.** Sections 76B.11 and 76B.13 use Riverstone's October–December 2025 order data — the same file Chapter 14 cleans and Chapter 72B interrogates — so the cost of a vague request is a real number rather than a warning. On that data "revenue" has four defensible answers spanning ₹6.25 crore, and "top five customers" has two correct answers with no names in common. > > **Levels and roles.** Each question carries a level and the roles that usually ask it. **Fresher:** screening calls and first-job interviews. **Mid:** one to three years in the role. **Senior:** lead or specialist rounds. **BA** business analyst · **DA** data analyst · **BI** BI developer. The basic-but-tricky questions come first; if you're aiming at your first BA job, start with section 76B.1 and the Fresher questions, then work through the rest.

76B.1 Basic-but-tricky BA questions

Q76B-007 · What's the difference between a BRD, an FRD, and an SRS?

Level: Fresher · **Roles:** BA, DA

Remember it as: *BRD is for the business, FRD is for the build, SRS is for the engineers, each one more technical and more detailed than the last.*

Answer in one line: A **BRD** (Business Requirements Document) captures the business need and goals, written for sponsors, executives, and finance, readable by someone who will never log in; an **FRD** (Functional Requirements Document) translates them into numbered functional requirements, process flows, business rules, and data definitions, written for developers and testers (detailed enough that two developers reading it build the same thing); an **SRS** (Software Requirements Specification) is the most technical of the three, detailed enough for a vendor to build from without talking to you, adding interfaces, non-functional requirements, data models, constraints, and error handling.

Tier	What to say
Passes	Knows all three acronyms exist, can't distinguish their actual audience or level of detail
Strong	The audience-and-detail-level distinction above, with a one-line example of the same requirement expressed at each level
Extra points	[+Edge cases] not every organization uses all three; some combine BRD and FRD into one document, or skip a formal SRS entirely and go straight to user stories (section 76B.3) for agile teams: naming that this varies by organization shows real experience, not just memorized definitions

Likely follow-ups: Which of the three would a project sponsor sign off on? Which would a developer actually build from?

Red flag: treating the three as interchangeable synonyms.

Learn it in: Chapter 25, section 25.7 (the three documents, and who reads each).

Q76B-008 · Is a user story the same thing as a requirement?

Level: Fresher · **Roles:** BA, DA, BI

Remember it as: *A user story is a conversation starter with acceptance criteria attached. A requirement can be a much heavier, more formal document. They're related, not identical.*

Answer in one line: Not exactly: a **user story** is a short, deliberately incomplete description of a need from a user's perspective ("As a [role], I want [goal], so that [benefit]"), meant to prompt a conversation and get refined collaboratively with the team; a **requirement** in the traditional sense is meant to be complete and unambiguous on its own, without needing further conversation to be understood.

Tier	What to say
Passes	Treats "user story" and "requirement" as interchangeable words for the same thing
Strong	The completeness distinction above: a user story is intentionally a placeholder for a conversation, not a finished specification
Extra points	[+Business] this is exactly why a user story needs acceptance criteria (Q76B-013) attached before it's actually "done": the criteria are what turn the conversation starter into something testable, closing the gap between the two

Likely follow-ups: In what kind of project would you use formal requirements documents instead of user stories, or both together?

Red flag: treating a bare user story ("As a user, I want a better dashboard, so that I can see my data") as if it were already a complete, actionable requirement.

Learn it in: Chapter 25, section 25.8 (use cases and user stories).

Rapid-fire, 76B.1

Roles: BA for every row; DA for Q76B-010 and Q76B-011.

#	Question	One-line answer	Extra point	Level · learn it in
Q76B-009	Use case vs. user story?	A use case describes a full interaction between an actor and a system, including alternate and exception flows, in more formal, structured detail / a user story is a short, single-sentence expression of a need	[+Business] use cases suit complex, multi-step system interactions; user stories suit iterative, conversation-driven agile work	Fresher · 25.8
Q76B-010	What's an assumption, in a requirements document, and why document it explicitly?	Something taken as true without confirmation, stated openly so it can be challenged or validated before it silently becomes a wrong foundation for the whole project	[+Assume] Chapter 69's Move 2 (state assumptions), the identical discipline applied to formal documentation instead of a live answer	Fresher · 25.7 (what a BRD holds); 69.3, Move 2
Q76B-011	What's the difference between "in scope" and "out of scope," and why does an explicit out-of-scope list matter?	In scope is what the project will deliver; out of scope is what it explicitly will not, stated up front to prevent scope creep and manage expectations before, not after, stakeholders assume otherwise	[+Business] an unstated out-of-scope item is the single most common source of a stakeholder feeling blindsided at delivery	Fresher · 25.7; 24.1 (question 3)
Q76B-012	What's a stakeholder register?	A structured list of everyone with an interest in the project, their role, their influence, and how and when they need to be communicated with: the power-interest grid written down as a table	[+Business] distinguishing high-influence, high-interest stakeholders (manage closely) from low-influence, low-interest ones (monitor only) is the core of practical stakeholder prioritization	Fresher · 24.3

76B.2 Requirements gathering

Q76B-001 · A stakeholder says "make the report better." How do you turn that into an actual requirement?

Level: Fresher · **Roles:** BA, DA, BI

Remember it as: "Better" is a feeling, not a requirement. Your job is to find the specific, checkable thing hiding behind the vague word.

Answer in one line: Ask specifically what problem the current report causes them, day to day, and work backward from that concrete pain point to a testable requirement, rather than guessing at what "better" might mean.

Worked example, a real elicitation dialogue:

> **Analyst:** "When you say 'better,' can you walk me through the last time the current report let you down?" > **Stakeholder:** "Last Tuesday I needed last week's numbers by region, and I had to open it in Excel and build a pivot table myself, it took twenty minutes." > **Analyst:** "So the requirement isn't 'better,' it's specifically: the report needs a region-level breakdown, viewable without any manual rework, available same-day."

Tier	What to say
Passes	Asks what "better" means, gets "faster, clearer", and writes that down with one follow-up question
Strong	Asks for a specific recent example of the pain, then restates it as a testable requirement in the stakeholder's own words
Extra points	[+Business] the fix here isn't asking more questions in general, it's asking for a <i>specific instance</i> , since people are far better at describing one real moment than defining an abstract concept like "better" [+Validate] restating the requirement back to the stakeholder in one sentence and getting a "yes, exactly" confirms it was captured correctly before any work starts

Likely follow-ups: What if the stakeholder can't recall a specific example? How would you prioritize this requirement against three other vague asks from different stakeholders?

Red flag: documenting "make the report better" as the requirement itself, or guessing at a specific fix with no elicitation at all.

Learn it in: Chapter 24, section 24.1 (turning an ask into a question) and Chapter 25, section 25.3 (the five moves from a vague ask to a written requirement).

Q76B-002 · What's the difference between a business requirement, a functional requirement, and a non-functional requirement?

Level: Fresher · **Roles:** BA, DA, BI

Remember it as: *Business requirement: why we're doing this. Functional requirement: what the system must do. Non-functional requirement: how well it must do it.*

Answer in one line: A **business requirement** states the underlying business goal ("reduce time spent reconciling orders"); a **functional requirement** states what the system must specifically do to support that goal ("the system shall flag any order with a quantity mismatch between the PO and the invoice"); a **non-functional requirement** states a quality constraint on how it does that ("the flag must appear within 2 seconds of the invoice being uploaded").

Tier	What to say
Passes	Can define one of the three, conflates the other two
Strong	All three definitions above, with a matched example showing how one business requirement can produce several functional and non-functional requirements underneath it
Extra points	[+Edge cases] a common real mistake: writing a functional requirement that's actually a design decision in disguise ("the system shall use a dropdown menu" describes a UI choice, not a requirement; "the system shall let a user select from a predefined list of statuses" is the actual requirement, implementation-neutral) [+Edge cases] names a data-specific non-functional requirement unprompted: freshness ("no more than two hours old during working hours") or reconciliation ("monthly revenue reconciles to the ERP to the rupee"), the ones that get dropped and then cause an incident

Likely follow-ups: Give a non-functional requirement that isn't about speed. Why does mixing requirement types in one document cause problems later?

Red flag: writing an implementation detail (a specific button, a specific dropdown) as if it were the requirement itself.

Learn it in: Chapter 25, section 25.5 (the three kinds, and the table of non-functional requirements specific to data).

Rapid-fire, 76B.2

Roles: BA for every row; DA for Q76B-004 to Q76B-006.

#	Question	One-line answer	Extra point	Level · learn it in
Q76B-003	What's a business rule, and how does it differ from a requirement?	A business rule is a statement of how the business works, independent of any system. Two kinds matter to a BA: constraints ("orders over ₹5 lakh need manager approval") and definitions ("active customer = a non-cancelled order in the trailing 90 days"). A requirement says what a system must do, often to enforce or compute one	[+Business] business rules usually outlive any one system; requirements are tied to the specific project building or changing that system	Mid · 24.2; 23.13 (the six-part template)
Q76B-004	What's elicitation, as a formal BA term?	The structured process of drawing out requirements from stakeholders, via interviews, workshops, document analysis, observation, or surveys, rather than requirements simply being handed over complete	[+Business] different elicitation techniques suit different situations: a workshop surfaces conflicting stakeholder views fast; direct observation catches things nobody thinks to mention because it's "just how we do it"	Fresher · 25.1 (the four verbs); 25.3
Q76B-005	Why interview more than one stakeholder for the same process?	Different roles see different parts of the process and often disagree about how it actually works, even for something they'd all claim to understand the same way	[+Evidence] a sales rep and a finance controller frequently describe the exact same order-approval process differently, and the gap between their descriptions is itself useful information	Fresher · 24.3; 25.4 ("the room starts talking")
Q76B-006	What's a requirements traceability matrix, in one sentence?	A table linking each requirement back to its business need and forward to the test case that verifies it, so nothing gets built without a reason or shipped without being checked	[+Business] it's the tool that answers "why are we building this" and "how do we know it worked" for every single requirement, at once	Mid · 25.7

76B.3 User stories and acceptance criteria

Q76B-013 · Write a complete user story with acceptance criteria for: "sales reps want to know which of their accounts are at risk of churning"

Level: Mid · **Roles:** BA, DA, BI

Remember it as: A user story with no acceptance criteria is a wish. Acceptance criteria are what turn a wish into something a developer can build and a tester can check off.

Answer in one line: The story states the role, the goal, and the benefit in one sentence; the acceptance criteria list the specific, testable conditions that must be true for the story to be considered done, including the actual rule, never a word like "recently" or "the defined threshold".

Worked example, the at-risk list Chapter 25 specifies (its FR-01 to FR-03):

> **User story:** As a sales rep, I want to see which of my accounts are at risk of not ordering again, so that I can call them before they go quiet. > > **Acceptance criteria:** > - **AC-1 (scope).** Given I am a logged-in sales rep, when I open the at-risk list, then I see only accounts where I am the assigned rep. > - **AC-2 (the rule).** Given an account with at least four days of orders whose time since its last order is more than 1.5 times its own average gap between orders, when the list is generated, then that account appears on it with an "At Risk" flag. > - **AC-3 (edge case).** Given an account with fewer than four days of orders, including one with a single order or none, when the list is generated, then it is not flagged and is counted separately as "too new to judge". > - **AC-4 (empty state).** Given I have no at-risk accounts, when I open the list, then I see a clear "no accounts currently at risk" message, not an empty or broken-looking screen. > - **AC-5 (reconciliation).** Given the list is refreshed, when the twelve-month revenue it shows for an account is compared with the ERP's figure for the same months, then the two agree to the rupee.

The rule in AC-2 isn't hypothetical. Run against Riverstone's 24 key accounts as of 31 December 2025, it flags 3 of them, and 7 are too new to judge (Chapter 25, section 25.6). That's why AC-3 exists: seven accounts that no rule is watching is a finding in itself.

Tier	What to say
Passes	A story plus one happy-path criterion, no edge case
Strong	The story plus Given/When/Then criteria for the scope, the actual rule, and at least one edge case
Extra points	[+Edge cases] the empty state (AC-4) is exactly the kind of edge case a strong BA adds unprompted: what happens when there's <i>nothing</i> to show is easy to forget and easy for a developer to leave broken if it's never specified [+Validate] a reconciliation criterion (AC-5), the fourth kind of criterion for data, and the one people new to data work miss [+Limits] ties the list to a real technical constraint: it's only as fresh as its data, which may be up to one working day old (Chapter 25's NFR-01), so a flag doesn't change the instant a customer orders

Likely follow-ups: How would you prioritize this story against three others in a sprint? What would make you split this into two smaller stories instead of one? What changes in AC-2 if the flag comes from a churn model's score instead of a rule?

Red flag: acceptance criteria so vague they can't actually be tested ("the dashboard should work well", or "flag accounts above the defined threshold" with no threshold).

Learn it in: Chapter 25, section 25.8 (Figure 25.3's story and its criteria, and the reconciliation criterion AC-5) and section 25.6 (the rule, and the query that counts it).

Rapid-fire, 76B.3

Roles: BA for every row; DA and BI for Q76B-014 and Q76B-016.

#	Question	One-line answer	Extra point	Level · learn it in
Q76B-014	What does "Given/When/Then" mean in acceptance criteria?	Given (the starting context) / When (the action taken) / Then (the expected result), a structured format (called Gherkin) for writing testable criteria	[+Business] this structure is what lets a QA engineer write a test case almost directly from the acceptance criteria with no further translation	Fresher · 25.8
Q76B-015	What's the INVEST mnemonic for a good user story?	Independent, Negotiable, Valuable, Estimable, Small, Testable	[+Edge cases] a story that fails "Independent" (it can't be built without another story finishing first) often signals it should be resequenced or the dependency made explicit	Mid · 25.8; 26.10
Q76B-016	Why should a user story avoid specifying a particular UI element?	The story should describe the need, not the implementation, so the team retains freedom to choose the best actual design; specifying a "dropdown" locks in a decision that isn't the BA's or the story's to make	[+Signpost] name the rule behind it: it's Q76B-002's line between a requirement and a design decision, applied to a story	Fresher · 25.8; 25.5
Q76B-017	What's a spike, in agile terminology?	A time-boxed research or investigation task, done specifically because a story can't be sized or written properly until an open technical question is answered first	[+Business] a spike is the honest alternative to guessing at a story's size or acceptance criteria when genuine uncertainty exists	Mid · 26.10; 25.2

76B.4 Process mapping

Q76B-018 · Map Riverstone's order-to-cash process at a high level, and explain what a swimlane adds that a simple flowchart doesn't

Level: Mid · **Roles:** BA, DA

Remember it as: A flowchart shows what happens. A swimlane shows what happens AND whose job it is, which is usually the more useful question in a real business process.

Answer in one line: A basic flowchart shows the sequence of steps; a **swimlane diagram** adds horizontal or vertical lanes, one per role or department, so each step visibly sits inside the lane of whoever's responsible for it, immediately surfacing handoffs between departments, which is usually exactly where a real process breaks down.

Worked example, order-to-cash for Riverstone's order 5001, swimlane by role (the numbers are Chapter 3's ten steps):

> **Customer lane:** 1 enquiry > **Sales lane:** 2 visit and agree terms → 3 quote → 4 order typed into the ERP > **Warehouse lane:** 5 stock check and reserve → 6 pick, pack, ship → 8 signed proof of delivery scanned > **Finance lane:** 7 invoice → 9 payment matched to the invoice → 10 monthly sales report

Tier	What to say
Passes	A flowchart with the right steps in order, with a note on who does each, but no lanes and no handoffs called out
Strong	The swimlane version above, correctly separating steps by which team owns them
Extra points	[+Business] the handoff <i>between</i> lanes (sales → warehouse, warehouse → finance) is almost always where delays and errors concentrate in a real process, an insight a plain flowchart hides entirely by not distinguishing ownership at all [+Evidence] on order 5001, five of the nine transitions cross a lane, and each crossing is where a data-quality problem is born: re-typed product codes at step 4, missing delivery dates at step 8 [+Edge cases] a real map would also show what happens on an exception path (a rejected order, a payment dispute), not just the clean happy path shown above

Likely follow-ups: Where in this process would you look first for a delay-reduction opportunity, and why? What would a BPMN version of this add beyond a plain swimlane diagram?

Red flag: a process map showing only the happy path, with no exception handling at all.

Learn it in: Chapter 25, section 25.4 (Figure 25.1, the swimlane of order 5001, and the lane table under it); the ten steps are Chapter 3, section 3.2.

Rapid-fire, 76B.4

Roles: BA for every row; DA for Q76B-021 and Q76B-022.

#	Question	One-line answer	Extra point	Level · learn it in
Q76B-019	What's a decision point (or gateway) in a process map?	A point where the process branches based on a condition (approved vs. rejected, in-stock vs. backorder), shown as a diamond in most notations	[+Edge cases] every decision point needs every branch mapped, including the "unhappy" ones; a map that only shows the approved path is incomplete	Fresher · 25.4
Q76B-020	What does BPMN stand for, and what does it add over a generic flowchart?	Business Process Model and Notation, a standardized symbol set (specific shapes for events, gateways, tasks, and messages between participants) that makes a process map unambiguous across different tools and readers	[+Business] standardization matters most when a process map needs to be handed to an external vendor or IT team who wasn't in the room when it was drawn	Mid · 25.4
Q76B-021	What's the difference between an "as-is" and a "to-be" process map?	As-is documents the process exactly as it currently, actually works, warts and all; to-be documents the proposed, improved version	[+Edge cases] skipping the as-is map and jumping straight to to-be is a common mistake: you can't credibly propose an improvement to a process you never actually documented as it really runs	Fresher · 25.4
Q76B-022	Why might the as-is process, once actually mapped, surprise the people who work in it every day?	People executing a process day to day often only know their own step; mapping the whole thing end to end frequently reveals workarounds, redundant approvals, or handoffs nobody had the full picture of before	[+Evidence] this is one of the most consistently reported "aha" moments in real BA work: the map itself is often the deliverable that creates the most value, before any redesign happens at all	Mid · 25.4

76B.5 SDLC and where a BA fits

Q76B-023 · Walk through the phases of the SDLC, and say specifically what a BA does in each

Level: Mid · **Roles:** BA

Remember it as: *A BA's role doesn't stop after requirements are written. It runs the whole length of the SDLC, in a different form at every stage.*

Answer in one line: Across the six phases, the BA does all of **Requirements** (elicitation, process mapping, gap analysis, writing and agreeing the requirements), defends the requirement in **Design** when the design quietly drops part of it, stays available in **Build** to answer the questions that would

otherwise be guessed at, writes or reviews the UAT plan and prepares the users in **Test**, facilitates UAT in **Deploy** while real users run it, then confirms sign-off and hands over, and in **Maintain** triages whether an issue is a genuine defect or a new requirement in disguise. The phase names vary by company; the sequence doesn't.

Tier	What to say
Passes	Names the SDLC phases correctly, describes the BA's role as "writing requirements" with no involvement in the other phases
Strong	The phase-by-phase involvement above, correctly identifying that a BA's role changes shape but doesn't disappear after requirements are handed off
Extra points	[+Business] the triage role at the maintenance stage (is this a bug, or a new requirement someone's calling a bug) is a genuinely common, underappreciated part of the job, and naming it specifically signals real experience rather than a textbook answer [+Edge cases] the BA facilitates UAT but doesn't drive it: if the BA drives the mouse, the test measures the BA, not whether the product meets the users' need

Likely follow-ups: How does a BA's role differ in a Waterfall project versus an Agile one? What happens when a developer's question during the build phase reveals the original requirement was actually ambiguous?

Red flag: describing the BA role as ending once requirements are signed off.

Learn it in: Chapter 25, section 25.2 (the six phases and the BA's part in each) and section 25.10 (who runs UAT).

Rapid-fire, 76B.5

Roles: BA for every row; DA and BI for Q76B-024 to Q76B-026.

#	Question	One-line answer	Extra point	Level · learn it in
Q76B-024	Waterfall vs. Agile, from a BA's perspective?	Waterfall: requirements are finalized once, up front, before development starts / Agile: requirements evolve iteratively across sprints, with the BA continuously refining the backlog	[+Business] most real organizations run a hybrid, not a pure version of either, worth naming rather than presenting the two as a strict binary	Fresher · 25.2; 26.10
Q76B-025	What's the difference between the product backlog and the sprint backlog, and who owns each?	The product backlog is everything not yet started, in priority order, owned and ordered by the Product Owner (the person accountable for what gets built and in what order); the sprint backlog is what the team pulled into this sprint, plus its plan, owned by the developers who'll do it. The BA feeds the product backlog with refined, testable stories	[+Edge cases] a BA and a Product Owner role can overlap significantly or be entirely separate people depending on the organization	Mid · 26.10; 25.2
Q76B-026	What's a Definition of Done, and why does a team need to agree on one explicitly?	A team-wide, explicit checklist of what "finished" means for any story (code reviewed, tested, documentation updated, and so on), agreed once so it doesn't get silently redefined story by story	[+Business] without an explicit Definition of Done, "done" quietly means something different to the developer, the BA, and the stakeholder, and that gap surfaces late, expensively	Mid · 26.10; 25.8
Q76B-027	What's a change request, and why does even a small one need a formal process?	A formally logged request to modify already-approved scope, requirements, or a signed-off deliverable; a formal process (however lightweight) protects against silent scope creep and keeps a record of who approved what	[+Business] the process doesn't need to be heavy for a small change, but skipping it entirely for "just this once" is exactly how scope creep starts	Mid · 25.11 and its Common mistakes ("agreeing a scope change in a corridor"); 63.9

76B.6 Gap analysis and UAT

Q76B-028 · What's a gap analysis, and walk through one for Riverstone's order-to-cash process

Level: Mid · **Roles:** BA, DA

Remember it as: *Gap analysis is just "where are we now" minus "where do we need to be," made explicit and specific enough to act on.*

Answer in one line: A **gap analysis** compares the current (as-is) state against the desired (to-be) state, identifying the specific differences that need to be closed and the root cause of each, each one becoming a candidate requirement or improvement initiative.

Worked example, the delivery step (step 8) of Riverstone's order-to-cash process:

> **As-is:** A driver brings back a paper proof of delivery. Someone at Bhiwandi Main scans it, emails it to finance, and changes the order to *Delivered* by hand, days later or not at all. The ERP holds no delivery date. > **To-be:** The delivery is recorded in the ERP when the customer signs, with the date and time. > **Gap:** Nothing records the delivery when it happens; the ERP hears about it later, from a scanned email. > **Root cause:** Not "the warehouse is slow". The only record of the delivery is a piece of paper at the customer's site: the driver has no device, so the fact travels back on the truck before anyone can type it in. > **Resulting requirement:** FR-14: the delivery status and date shall be recorded in the ERP within one hour of the proof of delivery being signed. It depends on NFR-09: a delivery recorded where there is no mobile signal shall reach the ERP within one hour of the signal returning, and none shall be lost.

Tier	What to say
Passes	Defines gap analysis correctly in the abstract, can't apply it to a concrete example
Strong	The full worked example above: as-is, to-be, the specific gap, its root cause, and the requirement it produces
Extra points	[+Business] the gap analysis doesn't stop at identifying the difference, it names the <i>root cause</i> (no device where the delivery happens, not "people are slow"), which is what actually makes the resulting requirement solve the real problem rather than a symptom of it [+Validate] asks "why" once more than feels necessary: telling the warehouse to scan faster would leave the date arriving days late anyway, so FR-14 can't work without NFR-09, the requirement that the recording survive a missing signal

Likely follow-ups: How would you prioritize this gap against several others found in the same analysis? What would you check before assuming a device for every driver is actually the right fix?

Red flag: a gap analysis that identifies a difference but never traces it to an actionable, specific requirement, or that names the symptom as the root cause.

Learn it in: Chapter 25, section 25.9 (Figure 25.4 is this row, and the ranking table scores it against two other gaps); the step itself is Chapter 3, section 3.2, and section 3.7 lists what goes wrong there.

Q76B-029 · What's UAT, who does it, and why can't developers or QA alone replace it?

Level: Fresher · **Roles:** BA, DA, BI

Remember it as: *UAT isn't "does the system work." It's "does the system actually do what the business needed," and only the business can really answer that.*

Answer in one line: **User Acceptance Testing** is the final validation stage, done by real business users (not developers or QA), checking whether the delivered system actually meets the original

business need, not just whether it's technically functioning correctly, since a system can pass every technical test while still failing to solve the real problem it was built for.

Tier	What to say
Passes	"UAT is when users test the system before launch" (correct, misses <i>why</i> it can't be skipped or delegated to QA)
Strong	The distinction above: QA verifies the system works as specified; UAT verifies the specification itself was actually right, which only the people with the original business need can judge
Extra points	[+Business] a system can pass 100% of its QA test cases and still fail UAT, because the test cases were built from a requirement that, it turns out, didn't fully capture what the business actually needed, exactly the kind of gap Q76B-001's elicitation discipline is meant to prevent upstream [+Validate] for a data product, UAT has a second job: reconcile at least one complete period against a source the business already trusts (last month's closed figures, the ERP's own report), at more than one level of aggregation, because a total can match while the breakdown doesn't

Likely follow-ups: What would you do if UAT reveals the original requirements were wrong, this late in the project? How would you structure a UAT test plan?

Red flag: treating UAT as a redundant, box-checking repeat of QA testing.

Learn it in: Chapter 25, section 25.10 (UAT, and the two halves of a data UAT plan).

Rapid-fire, 76B.6

Roles: BA for every row; DA and BI for Q76B-030.

#	Question	One-line answer	Extra point	Level · learn it in
Q76B-030	What's a UAT test case, and how does it differ from a QA test case?	Written from the business user's real workflow and language, checking business outcomes / QA test cases are written from the technical specification, checking system behavior	[+Validate] well-written acceptance criteria (Q76B-013) are already UAT test cases: they can be handed to a user as they stand	Mid · 25.10; 25.8
Q76B-031	What happens if UAT fails close to a planned go-live date?	The defect gets triaged: is it a genuine bug (fix and retest), a missed requirement (assess impact and possibly delay), or a misunderstanding of scope (clarify and possibly defer to a later phase)	[+Business] the temptation to launch anyway and "fix it after" needs to be weighed honestly against the real cost of the specific gap found, not treated as automatically acceptable or automatically a blocker	Senior · 25.10
Q76B-032	Who signs off on UAT completion?	Typically the business stakeholder or sponsor who owns the requirement, not IT or the project manager, since sign-off is specifically confirming the business need was met	[+Business] this sign-off is the formal moment accountability for "does this actually work for the business" transfers, worth taking seriously rather than as a formality	Mid · 25.10

76B.7 Stakeholder management and pushback

Q76B-033 · A stakeholder says "can you just change the number to make it look better?" How do you respond, live?

Level: Mid · **Roles:** BA, DA, BI

Remember it as: *Don't refuse and don't comply silently. Ask what's actually driving the request, then respond to that, not to the literal words.*

Answer in one line: Ask what's behind the request (are they worried the number looks bad to their own boss, do they suspect the number itself is wrong, is there missing context that would explain it) before either agreeing or refusing, since the literal request is almost never actually what they need.

Worked example, talked through live:

> "I'd say: 'I want to understand what's driving that, can you tell me more? If the number's wrong, I'll absolutely fix it, and I'd want to find the root cause. If it's right but looks concerning, I can add context that helps explain it rather than changing the number itself, since changing a correct number isn't something I can do.' Most of the time what they actually want is help explaining a real number, not a fabricated one, and naming that directly usually resolves it without an awkward confrontation."

Tier	What to say
Passes	Explains politely that the number is correct and can't be changed, and offers to double-check it, but doesn't ask what's driving the request or offer an alternative
Strong	The diagnostic question above, distinguishing "the number might be wrong" from "the number is right but needs context" before responding either way
Extra points	[+Business] offering a legitimate alternative (adding context, a footnote, a caveat) gives the stakeholder something real they can use, rather than leaving them with nothing after a flat refusal, which is often what actually causes the request to escalate or repeat

Likely follow-ups: What if the stakeholder insists after you've explained you can't change a correct number? How would you document this exchange, if at all?

Red flag: quietly changing a correct number to avoid conflict: altering a correct figure is an integrity failure, not a stakeholder-management choice. A defensive, unhelpful flat refusal with no attempt to solve the real underlying concern is the lesser failure.

Learn it in: Chapter 24, section 24.8 (the decision flow for "can you just change the number?", Figure 24.4).

Rapid-fire, 76B.7

Roles: BA, DA and BI for every row.

#	Question	One-line answer	Extra point	Level · learn it in
Q76B-034	Two stakeholders give you contradicting requirements. What do you do?	Don't silently pick one; bring both stakeholders together (or relay both positions clearly) so the conflict is resolved explicitly, with a documented decision, not quietly absorbed by the BA	[+Business] an undocumented, silently-resolved conflict tends to resurface later when the stakeholder who "lost" notices the outcome	Mid · 24.3 (RACI: who is accountable); 24.8; 25.11
Q76B-035	A powerful stakeholder wants a feature that conflicts with the actual data or user needs. How do you push back?	Lead with evidence, not opinion: show the specific data or user feedback that conflicts with the request, and frame it as a shared problem to solve together, not a personal disagreement	[+Signpost] bottom line up front, one message: Chapter 24's storytelling discipline applied to a difficult conversation instead of a report	Senior · 24.4; 24.8
Q76B-036	What's the risk of only communicating with the most senior stakeholder on a project?	Requirements end up shaped by whoever has the most authority to speak up, not necessarily whoever has the most accurate day-to-day knowledge of the actual process	[+Edge cases] the people closest to the work know the exceptions and the workarounds; that's Q76B-005's reason for interviewing more than one person	Mid · 24.3; 25.4
Q76B-037	How would you communicate a delay to a stakeholder who's expecting on-time delivery?	As early as possible, with the specific reason, the revised timeline, and what's being done about it, not a vague "it's taking longer than expected" with no further detail	[+Business] early, specific bad news is almost always received better than late, vague bad news	Fresher · 24.7

76B.8 Full scenarios, talked through live

Q76B-038 · Full scenario, talked through live: elicit requirements for automating Riverstone's payment matching

Level: Senior · **Roles:** BA, DA

What they're really testing: whether a full elicitation-to-requirement cycle can be run start to finish, live, not just described as a concept. The process is step 9 of order-to-cash: a finance assistant reads each bank statement line and works out which invoice it pays.

Talked through live, start to finish:

> "First, I'd ask finance to walk me through exactly how they match a payment today, step by step, including what they do when a line doesn't match, since the exception path usually matters more than the happy path. Say they tell me: they read each line on the bank statement, look up that customer's open invoices in the ERP, and record the payment against the one it pays; when the amount doesn't equal any open invoice, or the reference is as thin as 'SHARMA HW JAN', they work it out by eye or ask sales. I'd ask about the awkward cases by name: a customer paying one invoice in two instalments, or one transfer covering two invoices. I'd ask how often a line can't be matched cleanly, since if exceptions are rare, automation removes most of the work, and if they're common, it doesn't. Say it's about 15% of lines. At 15% exceptions, automation handles about 85% of lines unattended, which is worth building, as long as the other 15% go to a named person. I'd then ask what 'match' actually means precisely, exact to the paisa, or within some tolerance for rounding, since 'close enough' matching is a very different, harder requirement than exact matching. From there I'd draft a first-pass requirement: 'The system shall compare each imported bank line with the customer's open invoices and record a match when customer and amount agree within ₹1; any other line shall go to [named person] for review; every automatic match shall record the rule and time that produced it,' and take that back to finance to confirm before writing it up formally."

Extra-points moves demonstrated: **[+Clarify]** asked about the exception path, not just the happy path, unprompted. **[+Business]** asked for the exception frequency specifically because it changes the case for automating this at all, and said what 15% implies. **[+Scale]** asked for an audit trail and a named exception owner unprompted, the two additions most often forgotten when a person's job is automated. **[+Validate]** ended by proposing to confirm the draft requirement with the stakeholder before treating it as final, rather than writing it up unilaterally.

Likely follow-ups: How would you write the acceptance criteria for this requirement? What would change about your approach if this were a brand-new process instead of automating an existing manual one?

Red flag: drafting a requirement after asking only about the happy path, with no question about what happens when something doesn't match.

Learn it in: Chapter 25, section 25.3 (the five moves, run live here) and section 25.12 (specifying an automation: BR-05, BR-06, FR-15 to FR-17, and the three additions); the step is Chapter 3, section 3.2, step 9.

76B.9 Prioritisation: deciding what gets built

Every BA interview reaches "how do you prioritise?", and most candidates answer with a feeling. The frameworks below are what the question is fishing for, and the honest ranking of them matters more than reciting all four.

> **Beyond the book: the named prioritisation frameworks.** Chapter 25, section 25.12 gives this book's own method — rank on frequency, time per occurrence, cost of the errors caused, and effort and risk to automate — and Chapter 25, section 25.9 ranks gaps on hours saved × error weight ÷ build effort. Those are the frameworks to use on real work. **RICE is already introduced in Chapter 75,**

in the Beyond-the-book note under Q75-020, for prioritising product features; this section covers it again from the BA side, where the candidates are process changes rather than features, and adds the other three, which the book does not teach. Each gets one line here before the questions that use it. > > - **MoSCoW** sorts items into Must have, Should have, Could have and Won't have (this time). Its value is the fourth bucket. > - **RICE** scores $\text{Reach} \times \text{Impact} \times \text{Confidence} \div \text{Effort}$, giving a number you can sort on. > - **Kano** classifies features by how satisfaction responds: basic expectations, performance features, and delighters. > - **WSJF** (Weighted Shortest Job First) divides the cost of delaying something by how long it takes, so short valuable work goes first.

Q76B-039 · How do you prioritise a backlog when every stakeholder says their request is urgent?

Level: Mid · **Roles:** BA, DA, AE

Remember it as: *You do not prioritise. You make the cost of each choice visible and let the person accountable decide — then write down what they decided.*

Answer in one line: Score the requests on explicit, comparable criteria rather than arguing about urgency, present the ranking with its reasoning to the person accountable for the order (the product owner or sponsor), and record the decision — because a BA's job here is to make the trade-off legible, not to adjudicate between stakeholders.

The structure of a good answer, in four moves:

1. Separate urgent from important	A month-end report is urgent and date-bound; a data-quality fix is important and not. Both are real, and conflating them is how the important one never happens
2. Score on criteria everyone can see	Chapter 25's four (frequency, time, error cost, effort) or MoSCoW, but written down, so the argument moves from "mine matters more" to "is this number right?"
3. Escalate the decision, not the conflict	Take the ranking to the one accountable person. Two stakeholders cannot resolve each other's priority; a product owner can
4. Publish the result and the reason	Including what is <i>not</i> being done, so nobody discovers it in six weeks

Why move 1 is the one that earns the credit. "Everything is urgent" is almost always a mix of genuine deadlines and anxiety. Asking "what happens if this lands two weeks later?" sorts the list fast: a statutory filing has an answer, and a nice-to-have dashboard usually does not.

Tier	What to say
Passes	Mentions a framework (often MoSCoW) and says they would discuss it with stakeholders
Strong	The four moves, with explicit scoring, and the point that the BA makes the trade-off visible while the accountable owner decides
Extra points	+ [+Clarify] "what breaks if this is two weeks later?" as the single most useful question for sorting urgency from importance + [+Business] publish the not-doing list, because unstated deprioritisation is what destroys trust + [+Validate] revisit the ranking on a fixed cadence rather than only when someone complains, otherwise the loudest stakeholder effectively sets the order + [+Edge cases] a request with a hard external deadline (regulatory, audit, contractual) is not a priority call at all and should be taken out of the comparison

Likely follow-ups: Which framework would you actually use here, and why? What if the accountable owner will not decide?

Red flag: saying you would decide the priority yourself, or that you would "work on them all in parallel".

Learn it in: Chapter 25, §25.12 (ranking, with the four criteria); Chapter 24, §24.3 (stakeholder mapping) and §24.8 (pushback).

Q76B-040 · Explain MoSCoW, and tell me its weakness

Level: Fresher · **Roles:** BA

Remember it as: *Must, Should, Could, Won't. The method's value is entirely in the last two, which is why most teams get no value from it.*

Answer in one line: MoSCoW sorts requirements into **Must have** (the release fails without it), **Should have** (important but there is a workaround), **Could have** (desirable if time allows) and **Won't have this time** (explicitly excluded) — and its weakness is that without a hard limit on the Must bucket, everything migrates into it and the method stops sorting anything.

Bucket	The test that keeps it honest
Must	Name what specifically fails, and to whom, if this ships without it
Should	What is the workaround, and who does it in the meantime?
Could	Would you delay the release by a day for it? If yes, it is not a Could
Won't	Record it, with the reason and the date. This is the bucket that prevents the conversation recurring every sprint

The weakness, stated properly. MoSCoW is a sorting method with no forcing function. Nothing in it caps the Must bucket, so under pressure every item becomes a Must and the list is back where it started. Teams that get value from it add a constraint — a share of capacity, or a count — so that calling something a Must costs something.

The second weakness, worth mentioning if asked for more: MoSCoW ranks by necessity and says nothing about effort. A Must that takes six months and a Must that takes a day are the same

bucket, which is why it is usually paired with an effort estimate rather than used alone.

Tier	What to say
Passes	Expands the acronym correctly and gives an example of each
Strong	The four buckets plus the Must-bucket inflation weakness, and the fix of capping it
Extra points	+ [+Business] the Won't bucket is the real deliverable, because an explicit exclusion with a date and a reason stops the same request returning every sprint + [+Trade-offs] MoSCoW ignores effort entirely, so it pairs with an estimate rather than replacing one + [+Clarify] the Must test is "what fails, and for whom", which is a question a stakeholder can answer and "is this a must?" is not

Likely follow-ups: How would you cap the Must bucket in practice? When would you use RICE instead?

Red flag: not knowing what the W stands for, or treating it as "Would like to have".

Learn it in: Chapter 25, §25.12; Chapter 26, §26.10 (the backlog in priority order).

Q76B-041 · Walk me through RICE, and score two Riverstone requests with it

Level: Mid · **Roles:** BA, AE

Remember it as: $Reach \times Impact \times Confidence \div Effort$. The Confidence term is the one that makes it honest.

Answer in one line: RICE scores each item as **Reach** (how many people or events it affects in a period) times **Impact** (how much it helps each one, on a fixed scale) times **Confidence** (how sure you are, as a percentage) divided by **Effort** (person-time), producing a comparable number — and the Confidence multiplier is what stops a speculative high-impact guess outranking a known small win.

Worked, on two real Riverstone candidates from Chapter 25:

	A · Automate emailed order intake	B · Automate invoice matching (BR-05)
Reach	~173 orders/month (40/week — countable from the sales inbox)	~200 bank lines/month (assumed — nobody has counted)
Impact	3 (high: removes re-keying and its errors)	2 (medium: speeds the cash position)
Confidence	80% — the volume is countable and the process is observable	50% — the volume is a guess
Effort	3 person-months	2 person-months
RICE	$173 \times 3 \times 0.8 \div 3 = \mathbf{138}$	$200 \times 2 \times 0.5 \div 2 = \mathbf{100}$

A ranks above B, and **the reason it wins is Confidence, not Impact**. Order volume can be counted from the sales inbox this afternoon; the invoice-matching volume is nobody's measurement. RICE penalises the second for being a guess, and that is the property worth naming in an interview.

Be careful with the time-per-item figure, because this is where candidates invent data.

Chapter 3, section 3.7 gives the shape of the arithmetic — forty emailed orders a week at about six

minutes each is four hours a week, roughly two hundred hours a year — and the book is explicit that those are **round numbers for illustration, not a measurement**. Using them as though someone had timed the task is precisely the error Q76B-043 is about. The defensible version is to label the figure as illustrative and name the half-day of timing that would replace it.

The honest limits, which is what a strong answer adds:

- **The numbers are arguable.** Impact 3 against 2 is a judgement in numeric clothing, and two people will score it differently.
- **It invites false precision.** 138 against 100 looks decisive; it is not. RICE is for sorting a long list into bands, not for separating two close items.
- **Effort is the least reliable term** and it is the denominator, so an underestimate inflates the score most.

Tier	What to say
Passes	Expands RICE correctly and explains each term
Strong	Scores two real items, and identifies Confidence as the term that does the useful work
Extra points	+ [+Validate] label which inputs are measured and which are assumed, as the table above does, because an unlabelled estimate becomes a fact the moment it is in a spreadsheet + [+Trade-offs] false precision is the main risk: use the score for bands, not for close calls + [+Edge cases] Effort in the denominator means an optimistic estimate inflates the ranking most, so effort estimates should come from whoever will do the work + [+Business] re-score when confidence changes, since a cheap measurement that lifts Confidence from 50% to 90% can reorder the list more than any new feature + [+Signpost] Chapter 75's note under Q75-020 works RICE for product features and shows the arithmetic trap of multiplying by Effort instead of dividing — which inverts the ranking so the most expensive item wins

Likely follow-ups: How would you raise Confidence on item B? What would change if Effort were in weeks rather than months?

Red flag: scoring with no indication of which numbers are measured and which are invented — or quoting an illustrative figure as though it were timed.

Learn it in: Chapter 25, §25.12 and §25.9 (the book's own ranking methods); Chapter 75, Q75-020 (RICE for product features); Chapter 3, §3.7 (where manual work hides, and the illustrative arithmetic).

Q76B-042 · What is the Kano model, and when is it actually useful?

Level: Mid · **Roles:** BA

Remember it as: *Some features only hurt when missing, some help in proportion, and some delight. The first kind earns no credit for being there — only blame for being absent.*

Answer in one line: Kano classifies features by how user satisfaction responds to them — **basic expectations** (absence causes dissatisfaction, presence earns nothing), **performance features** (satisfaction rises with how well they are done) and **delighters** (unexpected, disproportionately appreciated) — and it is most useful for arguing that invisible work must be funded.

Category	Riverstone example	What it means for the plan
Basic expectation	The revenue figure is correct	Nobody thanks you. Getting it wrong is the only outcome anyone notices
Performance	The report arrives faster each month-end	More is better, roughly in proportion, so it is worth measuring
Delighter	The report flags the three customers whose orders dropped, unasked	Disproportionate appreciation for small effort

The one genuine use, and it is a good one. Kano is the argument for the work that earns no praise. Data correctness, reconciliation, validation rules — all basic expectations, all invisible when they work, all impossible to justify on a feature-value score. Kano gives you the language: *this is not a feature competing with features, it is the floor, and the project fails without it*. Chapter 72B's validation suite is exactly this kind of work.

The honest caveat: classifying a feature properly requires asking users a specific pair of questions (how would you feel if it were present; how would you feel if it were absent), and almost nobody does that survey. Used from the armchair it is a vocabulary rather than a method — which is fine, as long as you say so.

Tier	What to say
Passes	Names the three categories correctly
Strong	The three categories with examples plus the use case: funding invisible correctness work that no value score can justify
Extra points	+ [+Business] delighters decay into basic expectations over time, so today's differentiator is next year's floor and the category is not permanent + [+Validate] proper classification needs the functional/dysfunctional question pair asked of real users; without it, admit you are using the vocabulary rather than the method + [+Signpost] it is the strongest available argument for data-quality work, which otherwise loses every prioritisation contest to a visible feature

Likely follow-ups: Which Riverstone requirement is a basic expectation that is currently not met? How would you run the Kano survey?

Red flag: presenting Kano as a scoring method that produces a ranking. It produces categories, not an order.

Learn it in: Chapter 25, §25.5 (requirement types); Chapter 14, §14.10 (validation rules: checks that must return zero) and Chapter 72B, §72B.8.

Q76B-043 · You have no data to prioritise with and the stakeholder wants an answer today. What do you do?

Level: Mid · **Roles:** BA, DA

Remember it as: *Prioritise on what you can establish in an hour, say what you assumed, and name the one measurement that would change the answer.*

Answer in one line: Rank on the information available, label every assumption explicitly, give the ranking with a stated confidence, and identify the single cheapest measurement that would most change the order — because a decision today with named assumptions is more useful than a better decision next month, provided the assumptions are visible.

The three things that make this answer senior rather than reckless:

1. **Assumptions are labelled, not hidden.** "200 bank lines a month, assumed, not measured" in the table itself, the way Q76B-041 does it. 2. **A confidence is stated.** "I would hold A above B, but the gap is small and rests on an unmeasured volume." 3. **The next measurement is named and costed.** "Half a day counting last month's bank lines settles it" turns a weak answer into a plan.

What to avoid, and why. Refusing to rank is not caution, it is abdication — the decision gets made anyway, just without you. Equally, ranking with invented numbers and no labels is worse than refusing, because a number in a spreadsheet loses its caveat within a week. The defensible middle is a ranking whose assumptions travel with it.

Tier	What to say
Passes	Says they would gather more information first
Strong	Ranks anyway, labels the assumptions, states confidence, and names the cheapest measurement that would change the order
Extra points	+ [+Validate] the assumption labels must survive into whatever document circulates, because that is where they are usually stripped out + [+Business] "here is my ranking, here is what it rests on, here is the half-day that would confirm it" is a complete answer and the version a sponsor can act on + [+Clarify] ask what decision the ranking feeds: ordering next sprint needs far less rigour than committing an annual budget + [+Edge cases] if two items are genuinely indistinguishable on the evidence, say so and pick the cheaper one, because the cost of deciding further exceeds the difference

Likely follow-ups: How would you stop your labelled assumption becoming an unlabelled fact? What if the measurement would take a month?

Red flag: either refusing to give an answer, or giving one with confident invented numbers.

Learn it in: Chapter 24, §24.6 (the one-page memo, where assumptions go); Chapter 25, §25.12.

Rapid-fire, 76B.9

Roles: BA for every row, and DA or AE where noted.

#	Question	One-line answer	Extra point	Level · learn it in
Q76B-044	What is WSJF, and what problem does it solve that RICE does not?	Weighted Shortest Job First divides the cost of delay by job size, so it explicitly favours short valuable work; it solves sequencing under a fixed capacity, where RICE only ranks by value density	[+Trade-offs] cost of delay is harder to estimate than RICE's reach, which is why WSJF suits larger funded programmes rather than a request queue	Senior · 25.12
Q76B-045	What is a value-versus-effort matrix, and why is it so popular?	A two-by-two of value against effort, giving quick wins (high value, low effort), big bets, fill-ins and time sinks; popular because a stakeholder can read it in five seconds and argue with the placement, not the maths	[+Business] its real function is as a conversation device in a room, not as a ranking	Fresher · 25.12
Q76B-046	A stakeholder marks all twelve requirements as Must have. What now?	Impose the constraint MoSCoW lacks: ask them to fit the Musts inside the known capacity, or to order them, which forces the trade-off they avoided by marking everything a Must	[+Clarify] "if we could ship only four of these next month, which four?" gets an answer where "are these all musts?" does not	Mid · 25.12
Q76B-047	How do you prioritise data-quality work against a new dashboard?	By the cost of being wrong: a dashboard built on unvalidated data is negative value, because it produces confident decisions from bad numbers — so the quality work is a dependency, not a competitor	[+Signpost] Kano's basic-expectation category (Q76B-042) is the vocabulary for this argument	Mid · 14.10, 72B.8
Q76B-048	What is technical debt, and how should a BA treat it in a backlog?	Accumulated shortcuts that make future change slower; a BA's job is to get its cost expressed in terms the business understands — delivery time, defect rate, risk — so it competes on visible terms instead of being deferred forever	[+Business] debt framed as "the next change costs three weeks instead of three days" gets funded; framed as "refactoring" it does not	Mid · 26.10
Q76B-049	Two requirements depend on each other. How does that affect prioritisation?	A dependency overrides the score: the prerequisite inherits the priority of whatever depends on it, and hidden dependencies are the usual reason a high-priority item stalls mid-sprint	[+Validate] map dependencies before ranking, not after, or the ranking is unachievable	Mid · 25.12, 26.10
Q76B-050	Who owns priority: the BA, the product owner, or the stakeholders?	The product owner is accountable for the order; stakeholders supply the need and the value; the BA makes the trade-off visible and documents the decision	[+Edge cases] on a small data team these can be one person, so ask who signs off rather than assuming the titles exist	Fresher · 26.10

76B.10 Agile in practice: what a BA is actually asked

Chapter 26, section 26.10 teaches the vocabulary. This section tests it, and concentrates on the places where candidates who have only read about Agile come apart: estimation, velocity, and what happens when the ceremonies are theatre.

Q76B-051 • What are story points, and why not just estimate in hours?

Level: Fresher • **Roles:** BA, DA, AE

Remember it as: *Points size work relative to other work. Hours promise a date. Teams estimate size well and dates badly.*

Answer in one line: Story points express **relative size** — a 5 is bigger than a 3, roughly — rather than duration, which works because people compare two pieces of work far more reliably than they predict how long something takes, and because points absorb the differences between individuals that an hours estimate does not.

	Story points	Hours
What it measures	Relative size and complexity	Elapsed effort
Who it is comparable across	The team	One person, on a good day
What it becomes in a status meeting	Capacity for planning	A commitment, then a stick

The real reason teams moved to points. An hours estimate invites the question "so it will be done Thursday?", and the answer is treated as a promise. Points break that chain deliberately: they feed a planning conversation about how much fits in a sprint, not a delivery date for one item. Chapter 26 puts it plainly — points are relative size, not hours, and teams argue about this more than it deserves.

The honest part, which is what a strong answer includes. Points are routinely abused. Converting them back to hours ("our points are about a day each") restores exactly the problem they were meant to solve. Comparing velocity between teams is meaningless, because a point is defined only within one team. And an estimate presented to a sponsor as a commitment is a management failure, not an estimation technique failure.

Tier	What to say
Passes	"Points are relative size, not hours"
Strong	The relative-size argument, why it holds (people compare better than they predict), and the common abuses
Extra points	+ [+Trade-offs] a point has no meaning outside the team that set it, so cross-team velocity comparison is noise + [+Edge cases] a story nobody can size is a signal, not a failure: it needs a spike (Q76B-017) + [+Business] a sponsor needs a date, so the honest translation is a range from recent velocity with its uncertainty stated, not a point total + [+Clarify] ask what the estimate is <i>for</i> ; sequencing needs far less precision than a contractual date

Likely follow-ups: How would you give a sponsor a date from points? What is planning poker?

Red flag: "a point is about a day", which collapses the distinction entirely.

Learn it in: Chapter 26, §26.10 (story points, the backlog, and the honest part).

Q76B-052 · What are velocity and a burndown chart, and how can both mislead?

Level: Mid · **Roles:** BA, AE

Remember it as: *Velocity is how many points a team finished recently. A burndown is work remaining over time. Both are planning aids, and both become targets the moment someone reports them upward.*

Answer in one line: **Velocity** is the points completed per sprint, averaged over recent sprints, used to forecast how much fits in the next one; a **burndown chart** plots remaining work against time within a sprint or release; and both mislead as soon as they become performance measures, because a team can raise either by inflating estimates rather than delivering more.

How each one actually fails:

	How it misleads
Velocity as a target	Points are self-assigned, so the cheapest way to raise velocity is to estimate higher. The number rises, the output does not
Velocity across teams	A point is defined inside one team. Comparing two teams' velocity compares two different units
A flat burndown	Usually means work is being started and not finished, which looks like effort and delivers nothing. A WIP limit addresses it; a stern conversation does not
A cliff at the end	Everything closes on the final day, which means items were not genuinely done until then and the chart had no predictive value all sprint

The useful framing. Both are instruments for the team's own planning. A flat burndown is informative *to the team* on day four, when they can still act. The same chart in a steering-committee pack is a compliance artifact, and that is the point at which the measurement starts corrupting the thing it measures.

Tier	What to say
Passes	Defines both correctly
Strong	Defines both and explains the specific failure mode of each, including estimate inflation
Extra points	+ [+Business] name the mechanism: because the team sets the points, any target on velocity is a target on the team's own ruler + [+Validate] a flat burndown read on day four is a useful signal; read in a monthly report it is history + [+Edge cases] velocity is meaningless for the first few sprints of a new team and after any change in membership + [+Trade-offs] throughput (items finished per week) is harder to game than velocity, which is one reason Kanban teams prefer it

Likely follow-ups: What would you use instead to reassure a sponsor? What does a burndown going up mean?

Red flag: treating rising velocity as evidence of improvement.

Learn it in: Chapter 26, §26.10 (sprints, the board, and WIP limits).

Q76B-053 · What happens in backlog refinement, and what is the BA's job in it?

Level: Mid · **Roles:** BA, AE

Remember it as: *Refinement is where a vague item becomes something a team can size. The BA's job is to arrive having already removed the ambiguity.*

Answer in one line: Refinement (also called grooming) is the recurring session where upcoming backlog items are clarified, split, given acceptance criteria and sized — and the BA's job is to do the elicitation *before* the meeting, so the team spends it sizing work rather than discovering what the work is.

What a refined item has, which is the checklist worth knowing:

- A clear user-facing outcome, in the story form Chapter 25 §25.8 teaches
- **Acceptance criteria**, testable, in Given/When/Then if the team uses it
- Dependencies and assumptions named
- Small enough to finish inside one sprint — otherwise split
- Passes **INVEST** (Q76B-015)

Where it goes wrong, and the BA's part in it. The failure mode is a refinement session that becomes a requirements workshop: the team sits through an hour of discovery that two conversations beforehand would have settled. That is a BA failure, not a process failure. The second failure mode is the opposite — a BA who writes complete solutions and brings them for ratification, which wastes the team's design judgement and produces stories nobody owns.

The line to hold: the BA brings the problem fully understood and the solution genuinely open.

Tier	What to say
Passes	Describes refinement as clarifying and estimating upcoming items
Strong	The refined-item checklist, and the BA's job as doing the elicitation beforehand
Extra points	+ [+Business] a refinement session spent on discovery is the most expensive possible way to do elicitation, because the whole team is in the room + [+Clarify] bring the problem closed and the solution open, which is the distinction between a prepared BA and one who writes designs + [+Edge cases] an item that cannot be sized after one refinement needs a spike rather than a longer argument + [+Validate] splitting is the main output: an item too big for a sprint is not a priority decision, it is a writing problem

Likely follow-ups: How would you split a story that is too big? How often should refinement happen?

Red flag: describing it as a meeting where the BA presents finished requirements for approval.

Learn it in: Chapter 26, §26.10; Chapter 25, §25.8 (stories and criteria).

Q76B-054 · What is an MVP, and what is the most common way people get it wrong?

Level: Mid · **Roles:** BA, DA

Remember it as: *Minimum viable means the smallest thing that is genuinely usable and teaches you something. Not the smallest thing you can ship.*

Answer in one line: A minimum viable product is the smallest version that delivers real value to a real user and answers the riskiest open question — and the common error is dropping the **viable**, shipping something incomplete that nobody can use, which teaches nothing because nobody uses it.

The two failure modes, which is the whole answer:

	What it looks like	Why it fails
Minimum without viable	A dashboard with three of the eight numbers anyone needs	Nobody adopts it, so you learn nothing and the follow-up funding disappears
Viable without minimum	Six months of build before anyone sees it	By the time you learn anything, the cost of being wrong is already sunk

A concrete Riverstone version. For the order-intake automation, an MVP is not "parse 30% of emails". It is **handle the one highest-volume order format end to end, for one branch, with everything else routed to a person** — genuinely usable by that branch, and it answers the risky question of whether automated parsing is accurate enough to trust. Chapter 58, section 58.7 is exactly why that question is the risky one: the pipeline it builds reaches an 88% straight-through rate, which looks like success, while **19% of the orders it loads unattended are quietly wrong**. The 12% it hands to a person, with reasons, is the safe part. **An MVP that surfaced those two percentages early is worth more than one that shipped more coverage** — and Chapter 58's own point is that only one of the two numbers ever reaches a business case.

Tier	What to say
Passes	Defines MVP as the smallest shippable version
Strong	Both failure modes, and the point that the MVP's job is to answer the riskiest question, not merely to be small
Extra points	+ [+Validate] define in advance what the MVP is meant to teach and what result would stop the project, otherwise it is just phase one + [+Business] "minimum" is negotiated against viable , and dropping viability to hit a date produces something that cannot be learned from + [+Edge cases] for internal data products the viability bar is often <i>correctness</i> , not feature count, because a wrong number destroys adoption faster than a missing one + [+Signpost] Chapter 58, §58.7's 88% against 19% is the measured case for making accuracy the MVP's question

Likely follow-ups: What would make you stop the project after the MVP? How is an MVP different from a proof of concept?

Red flag: an MVP defined purely by what fits in the time available.

Learn it in: Chapter 25, §25.6 (the four things a data team builds); Chapter 58, §58.7 (the two rates, and the cost table).

Rapid-fire, 76B.10

Roles: BA for every row.

#	Question	One-line answer	Extra point	Level · learn it in
Q76B-055	Epic, story, task, bug — what is the difference?	Sizes and kinds of work: an epic groups many stories, a story is a user-visible piece of work, a task is a piece of a story, a bug is something that already does not work as specified	[+Edge cases] the distinction is per-team convention in the tracker, so ask rather than assume	Fresher · 26.10
Q76B-056	What is planning poker?	Everyone sizes an item privately and reveals simultaneously, so the discussion is driven by the disagreement rather than anchored on whoever speaks first	[+Business] the estimate is the by-product; the conversation that the disagreement forces is the value	Fresher · 26.10
Q76B-057	What is t-shirt sizing and when would you use it?	Sizing as S/M/L/XL rather than numbers, used early when items are too vague for points and precision would be false — often for a roadmap rather than a sprint	[+Trade-offs] it resists being summed, which is both its weakness and its defence against being treated as a commitment	Fresher · 26.10
Q76B-058	What is a WIP limit, and why does it make a team faster?	A cap on items in progress at once, which makes finishing beat starting; without it work accumulates half-done, and nothing half-done delivers any value	[+Signpost] this is the fix for the flat burndown in Q76B-052	Mid · 26.10
Q76B-059	What does a BA do in a daily standup?	Unblocks: answers requirement questions, chases a stakeholder decision, clarifies acceptance criteria — not report status upward, which is the antipattern that turns the meeting into a status meeting for a manager	[+Business] if the standup is reporting to a manager, the team stops saying what is actually blocked	Fresher · 26.10
Q76B-060	Product Owner, Scrum Master, BA — how do they differ?	The PO is accountable for what gets built and in what order; the Scrum Master helps the team run the process; the BA does the analysis that makes a requirement buildable — and on a small data team one person often holds two of the three	[+Clarify] ask which of these roles actually exists at the company before describing your fit	Fresher · 26.10
Q76B-061	Your team's retrospectives have stopped changing anything. What would you do?	Make one action owned, dated and small, and check it at the next retro — a retrospective with no tracked action is a venting session, and the honest fix is one change that visibly happens	[+Business] Chapter 26 names this directly: the practices are worth what the team makes of them	Mid · 26.10

#	Question	One-line answer	Extra point	Level · learn it in
Q76B-062	How do you handle a requirement that arrives mid-sprint?	Not into the sprint by default: assess whether it is genuinely urgent, and if it is, something comes out to make room, with the PO deciding what — because silently absorbing it is how a sprint commitment becomes meaningless	[+Signpost] a formal change request (Q76B-027) applies if scope, cost or date move	Mid · 26.10
Q76B-063	Waterfall for a data warehouse migration, or Agile?	Usually a mix: the data model and the cutover need up-front design because they are expensive to change, while the reports and dashboards built on top suit iteration — and saying "it depends, here is the split" beats picking a side	[+Trade-offs] the test is cost of change: high-cost-to-change decisions earn more up-front analysis	Senior · 25.2, 26.10

76B.11 The ambiguity drill

This section has no counterpart in any other bank, and it is the one to practise out loud.

Every other question bank in Part VIII asks you to produce an answer. **This one asks you to produce the questions**, because that is the actual BA skill and it is the thing interviewers test when they hand you a vague request and watch what you do with it. Chapter 24, section 24.1 and Chapter 25, section 25.3 teach the method; this is the drill.

How each drill works. You get a request exactly as a stakeholder would send it. Before reading on, say out loud the questions you would ask before writing any code or any requirement. Then compare. Each drill names the questions that matter, says **which ones change the number most**, and gives the one-line reply you would actually send.

Why the stakes are real here. The drills use Riverstone's October–December 2025 order data, the same file Chapter 14 cleans and Chapter 72B interrogates, so the cost of guessing is measurable rather than rhetorical. On that file, "revenue" has four defensible answers spanning **₹6.25 crore**, and "top five customers" has two correct answers with **no names in common**. A BA who does not ask is not saving time; they are picking one of several different answers at random and presenting it as the answer.

The one rule. Ask the questions that change the answer, not every question you can think of. A stakeholder who gets eleven questions back stops replying. Three sharp ones get an answer the same day, and the skill is knowing which three.

Q76B-064 · "Can you send me last month's revenue?"

Level: Fresher · **Roles:** BA, DA

Remember it as: "Revenue" is not a number. It is a family of numbers, and the gap between its members is usually larger than anything you will be asked to explain.

Answer in one line: Ask whether they want it **gross or net of discount** and whether **cancelled orders are in or out**, because those two choices alone move the Q4 figure by about ₹2 crore and nothing on the output would tell the reader which version they received.

The questions, in the order that matters:

	Question	Why it matters here
1	Gross or net of discount?	Riverstone's discounts run to 12%. This is the single biggest fork
2	Are cancelled orders in or out?	Worth ₹1.87 crore on this quarter alone
3	Delivered only, or everything not cancelled?	Pending and shipped orders are real revenue but not yet realised
4	Which month, and dated how — order date or entry date?	The two can disagree at a month boundary
5	What is the number for?	A board pack and a sales commission calculation want different answers, legitimately

The four answers, all correct, all different. Measured on the Riverstone Q4 file:

Definition	Figure
Gross, before discount	₹45.89 cr
Net of discount, all lines	₹44.26 cr
Net, excluding cancelled	₹42.39 cr
Net, delivered only	₹39.63 cr

₹6.25 crore between the widest pair — 16%. Nobody receiving a single figure can tell which of the four they are holding, and all four would be defended by someone as the obvious meaning.

Questions 1 and 2 are the ones to ask. Together they account for almost the whole spread.

Questions 3 and 4 are refinements, and question 5 is the one that makes the other four unnecessary next time, because knowing the purpose usually determines the definition.

The reply to send:

> Happy to — two quick things so I send the right number. Do you want it **net of discount** (I'd suggest yes), and should **cancelled orders be excluded**? Excluding them changes Q4 by about ₹1.9 crore, so

it matters. If it's for the board pack I'd default to net, excluding cancelled, and label it that way on the slide.

Note what that reply does: it asks two questions, quantifies why one of them matters, proposes a default so the stakeholder can simply say "yes", and commits to labelling the definition on the output. Chapter 24's pyramid principle in miniature.

Tier	What to say
Passes	Asks which month and whether to include cancelled orders
Strong	Asks the gross/net and cancelled questions first, having recognised they dominate, and proposes a default rather than only asking
Extra points	+ [+Business] quantify the fork in the question itself — "about ₹1.9 crore" — because that is what makes a stakeholder actually answer + [+Validate] put the definition on the output, every time, so the number travels with its meaning + [+Clarify] ask what decision it feeds, which usually settles the definition without further discussion + [+Edge cases] "last month" is itself ambiguous in the first days of a month, and with a reporting calendar that is not the Gregorian one

Likely follow-ups: Which definition would you default to, and why? What if they say "just the normal one"?

Red flag: sending a number. Any number.

Learn it in: Chapter 24, §24.1 (turning an ask into a question); Chapter 25, §25.3; Chapter 23, §23.13 (defining a metric so two teams get the same number).

Q76B-065 · "How many orders did we get last quarter?"

Level: Fresher · **Roles:** BA, DA

Remember it as: *Ask what one row means before you count rows. "Order" and "order line" differ by 80% on this data.*

Answer in one line: Ask whether they mean **orders or order lines** before counting anything, because on this data the two answers are 14,372 and 25,832 — an 80% difference with no visible symptom.

The questions:

1. **Do you mean orders, or order lines?** An order with three products is one order and three lines.
2. **Cancelled orders included?** 3. Dated by order date or entry date? 4. Is a repeat customer's second order a separate order? (Almost always yes — but ask once.)

The three answers, measured:

Definition	Count
Order lines	25,832
Distinct orders	14,372
Distinct orders, excluding cancelled	13,777

Question 1 is the whole question. Reporting 25,832 when the answer is 14,372 overstates by **80%**, and nothing about the figure reveals the error — it is a plausible number, on a plausible chart, in a plausible report. This is the grain question of Chapter 72B, Q72B-005, arriving as a stakeholder request rather than as a file.

The reply:

> Quick check — do you want **orders** or **order lines**? There were 14,372 orders made up of 25,832 lines last quarter, so the two answers look very different. I'd assume orders unless you're sizing warehouse picking work.

The last clause is the valuable part: it names the one situation in which the other answer is the right one, which shows the question was not pedantry.

Tier	What to say
Passes	Asks whether cancelled orders count
Strong	Asks the grain question first, gives both figures, and names when each is the right one
Extra points	+ [+Business] give both numbers in the reply, since the stakeholder usually recognises the one they meant on sight + [+Signpost] the warehouse wants lines and finance wants orders, so neither is wrong and the grain belongs in the report title + [+Validate] $25,832 \div 14,372$ is 1.80 lines per order, a ratio worth stating because it makes the difference between the two intuitive

Likely follow-ups: What is the grain of the table you would query? How would you label the figure?

Red flag: counting rows without establishing what a row is.

Learn it in: Chapter 25, §25.3; Chapter 72B, §72B.2 (Q72B-005); Chapter 28, §28.8 (the grain discipline).

Q76B-066 · "Who are our top five customers? I need it for the loyalty programme."

Level: Mid · **Roles:** BA, DA

Remember it as: "Top" needs a measure. On this data, top-by-revenue and top-by-frequency share nobody.

Answer in one line: Ask **what measure defines "top"** and **what the programme is meant to reward**, because top-five-by-revenue and top-five-by-order-count share no customers at all on this

quarter's data.

The questions:

1. **Top by what — revenue, order count, or margin?** 2. Over what period? 3. Cancelled orders included? 4. **What makes a customer valuable to the loyalty programme?** — the question behind the question

The measured result, and it is the strongest thing in this section:

Ranked by	Top three
Revenue	Golden Wholesale Surat · Tulsi Exports · Coconut Grove Wholesale
Order count	Rainbow Packaging · Silk Route Depot Bengaluru · Saffron Agencies

The two top-five lists have zero names in common. Not a different order — a completely different set of customers. Both lists are correct. One is "who spends the most", the other is "who buys most often", and a loyalty programme built on the wrong one rewards the wrong people and the error is invisible, because any list of five large customers looks right.

This is why question 4 matters more than questions 1 to 3. A loyalty programme usually wants to reward and retain *frequent* behaviour, which points to order count — or to a blend, since a once-a-year customer placing a huge order is also worth keeping. The stakeholder asked for "top five"; what they need is a definition of valuable, and that is a conversation, not a query.

The reply:

> Before I pull it — top five by **revenue** and top five by **number of orders** turn out to have no customers in common on last quarter's data, so the choice decides who gets into the programme. Which behaviour is the programme meant to reward? If it's repeat buying I'd use order count, and I'd suggest we look at both lists together before deciding.

Tier	What to say
Passes	Asks "top by revenue or by volume?"
Strong	Asks the measure question and the purpose question, and recognises the measure choice decides the programme's membership
Extra points	+ [+Validate] the zero-overlap finding is worth producing <i>before</i> the conversation, because it converts an abstract definitional question into an obvious one + [+Business] margin is usually the right measure and the hardest to get, so say that rather than silently substituting revenue + [+Edge cases] a single very large one-off order can put a customer in the revenue top five with no ongoing relationship at all, which is exactly who a loyalty programme should not target + [+Clarify] ask whether the programme is meant to reward past behaviour or change future behaviour, which are different analyses

Likely follow-ups: How would you blend the two measures? What if margin data does not exist?

Red flag: picking revenue because it is the easiest column to sum.

Learn it in: Chapter 24, §24.1; Chapter 23, §23.13; Chapter 25, §25.3.

Q76B-067 · "What is our average order value?"

Level: Mid · **Roles:** BA, DA

Remember it as: *Average of what, over what, and is the mean even the right statistic?*

Answer in one line: Ask whether the average is **per order or per line**, and report the **median alongside the mean**, because the grain changes the figure by 80% and the mean-median gap is itself a finding about the distribution.

The questions:

1. **Per order, or per order line?** 2. Mean or median? 3. Cancelled orders in or out? 4. Net or gross?

Measured:

Mean per order line	₹17,133
Mean per order	₹30,794
Mean per order, excluding cancelled	₹30,767
Median per order	₹26,250

Two separate forks, and both matter. Line against order is **80%**, the same grain error as Q76B-065. Mean against median is **₹4,544**, about 17%, and that gap is information rather than noise: it says the distribution is right-skewed, so a few large orders are pulling the mean up.

The part that earns the credit. If the figure is for a target — "we want to raise average order value" — the median is usually the better measure, because the mean can be moved by one unusually large order without anything about typical customer behaviour changing. Saying that unprompted is the difference between answering the question and being useful.

The reply:

> Per order it's ₹30,794 on the quarter, and the median is ₹26,250 — the gap means a few big orders pull the average up. If this is going into a target I'd suggest the median, and I'd confirm you want it per order rather than per line (per line it's ₹17,133).

Tier	What to say
Passes	Asks whether to include cancelled orders
Strong	Identifies both forks — grain, and mean versus median — and gives the figures
Extra points	+ [+Validate] report the mean and median together; the gap between them is a finding, not a formatting choice + [+Business] for a target, recommend the median and say why, because a mean target can be hit by one large order + [+Edge cases] an average over a mixed customer base hides more than it shows, so segment by customer type before anyone sets a target on it

Likely follow-ups: Why is the distribution skewed? How would you segment it?

Red flag: reporting a mean with no reference to the distribution's shape.

Learn it in: Chapter 23, §23.13; Chapter 15, §15.5 (distributions: histograms and box plots).

Q76B-068 · "The December number in your report doesn't match the one finance sent me."

Level: Mid · **Roles:** BA, DA, AE

Remember it as: *Two numbers that disagree are usually two definitions. Find the definition difference before you look for a bug.*

Answer in one line: Get finance's **exact figure** and compare **definitions** — gross or net, which statuses, which date basis, which period boundary — before looking for a bug, because the discrepancy is definitional nine times in ten.

The questions, and the order is the lesson:

1. **What is finance's figure, exactly?** Get the number before anything else. 2. **What definition does theirs use** — net or gross, cancelled in or out, invoiced or ordered? 3. **What date basis** — order date, invoice date, entry date, payment date? 4. **What period boundary** — calendar December, or a reporting month that ends on a different day? 5. Only then: is either figure actually wrong?

Why this order. Nine times in ten the discrepancy is definitional, and investigating the data first wastes a day and still ends with the definitional question. Working the definitions first either explains the gap or, by elimination, turns it into a genuine data question with the easy explanations ruled out.

A measured example from this file. December net revenue is **₹9.11 crore** dated by order date. Date the same lines by their entry timestamp in **UTC** rather than Indian time and December becomes **₹9.10 crore** — a gap of about **₹1.42 lakh** from nothing but the time zone, with no bug anywhere and both figures correctly computed. That is small enough to be dismissed as rounding and large enough to fail a reconciliation to the rupee, which is the worst combination. Chapter 72B, Q72B-027 measures the underlying cause: on 1,604 rows the UTC date is not the Indian date.

The reply:

> Can you send me finance's exact figure? Mine is ₹9.11 crore, which is net of discount, all statuses, dated by order date, calendar December. Most of these differences turn out to be one of those four choices rather than an error — if theirs is invoiced rather than ordered, or dated on payment, that would explain it. I'll reconcile and come back with the decomposition.

Tier	What to say
Passes	Says they would check their own calculation
Strong	Works the definitions first, states their own figure's full definition unprompted, and treats "which is wrong" as the last question
Extra points	+ [+Validate] state your own definition in full before asking about theirs, which often resolves it in one message + [+Business] never tell a stakeholder the other team is wrong before you know; a definition difference framed as an error damages a relationship you need + [+Edge cases] the time-zone case above produces a real, unexplainable-looking gap with no bug, which is why you ask about date basis explicitly + [+Signpost] decompose the residual rather than accepting a tolerance, as Chapter 72B's Q72B-069 argues

Likely follow-ups: What if both are right and the definitions are both reasonable? How would you prevent this recurring?

Red flag: assuming the other team is wrong, or starting by re-running your own query.

Learn it in: Chapter 14, §14.11 (reconciling and documenting every decision); Chapter 72B, §72B.8; Chapter 24, §24.8.

Q76B-069 · "Just add a column for last month to the dashboard."

Level: Mid · **Roles:** BA, DA, AE

Remember it as: *"Just add a column" is a request to change a definition, a refresh schedule and a layout at once. None of those is one column.*

Answer in one line: Ask whether "last month" means the **previous calendar month or a rolling 30 days**, and **what it will sit beside**, because a complete period placed next to an incomplete one reads as a decline every month until the last day of it.

The questions:

1. **Last month relative to what — a rolling 30 days, or the previous calendar month?** These differ every day of the month except one. 2. **Comparing to what?** A bare "last month" column invites a comparison nobody specified: last month against this month-to-date is comparing a complete period with an incomplete one. 3. Does it need to update as this month progresses? 4. Which existing metrics get the column — all of them, or specific ones? 5. Does anything have to come off the dashboard to make room?

The trap, which is question 2. Putting last month beside month-to-date produces a column that looks lower every single month until the final day, and someone will read it as a decline. The fix is either a like-for-like comparison (same number of days elapsed) or an explicit label. This is the most common way a correct dashboard misleads, and it arrives disguised as a one-column request.

The reply:

> Two things: do you mean the previous **calendar** month or a rolling 30 days? And should it sit next to month-to-date? If so I'd compare the same number of elapsed days, otherwise the current month always looks worse until the last day of it. Small change either way — I'd rather get the comparison right than add it twice.

Tier	What to say
Passes	Asks whether it means calendar month or rolling 30 days
Strong	Identifies the incomplete-period comparison trap and proposes the like-for-like fix
Extra points	+ [+Business] "just" is a signal to slow down, not speed up: it usually means the requester has not thought about the comparison + [+Validate] a partial period beside a complete one needs an explicit label on the dashboard itself, not in a footnote nobody reads + [+Edge cases] a rolling window and a calendar month agree on exactly one day a month, which is when this gets signed off and when nobody notices + [+Clarify] ask what decision the comparison supports, because a trend question and a target question want different columns

Likely follow-ups: How would you label an incomplete period? What if the dashboard is already full?

Red flag: adding the column as asked.

Learn it in: Chapter 24, §24.1; Chapter 15, §15.12 (misleading charts, and how not to make them); Chapter 23, §23.13.

Rapid-fire, 76B.11 · more vague asks, drilled

For each one: the question that matters most, before anything is built. Roles: BA and DA for every row.

#	The request	The question to ask first	Why it dominates	Level · learn it in
Q76B-070	"Make the report real-time."	What decision needs to be made faster than it is now?	"Real-time" almost always means "fresher than daily", and the cost difference between hourly and true streaming is enormous; the decision cadence sets the requirement	Mid · 25.5, 50.1
Q76B-071	"Which branch is performing best?"	Best by total revenue, or per order, or growth?	On this data Mumbai HO leads on both revenue (₹15.73 cr) and revenue per order, but revenue per order is nearly flat across all four branches (₹30,241–₹30,994), so the branches differ in volume , not in order value — which is the actual finding and changes what "performing" should mean	Mid · 23.13, 25.3
Q76B-072	"Can you make this dashboard load faster?"	How slow is it now, and how fast is fast enough?	Without a measured baseline and a target there is no requirement, only an open-ended optimisation; this is a non-functional requirement and needs a number	Mid · 25.5
Q76B-073	"Everyone should be able to see this."	Everyone meaning whom, and is any of it restricted?	Customer and salary data have access rules; "everyone" is almost never meant literally, and getting it wrong is a disclosure rather than a bug	Mid · 25.5, 63.9
Q76B-074	"We need to track customer churn."	What counts as churned, and over what window?	For a wholesaler with irregular ordering there is no login to stop: churn has to be defined as an absence of orders over some period, and that period is the entire analysis	Senior · 23.13, 25.3
Q76B-075	"Add the data from the new system too."	Do the two systems define a customer and an order the same way?	Joining two sources with different definitions produces a table where no row means one thing; the key and definition reconciliation is the work, not the load	Senior · 14.8, 25.11

76B.12 Tools, artifacts and estimation

Q76B-076 · Which BA tools have you used, and what for?

Level: Fresher · **Roles:** BA

Remember it as: *Name the tool, then the artifact you produced in it, then the decision it supported. A list of tools answers nothing.*

Answer in one line: Answer in the shape **tool** → **artifact** → **outcome** rather than listing software, because every candidate lists the same four tools and the interviewer is trying to find out whether you have actually produced anything in them.

Tool	What a BA produces in it	What it is for
Jira	Epics, stories with acceptance criteria, linked issues	The backlog of record, and the trace from request to code
Confluence (or any wiki)	BRD, FRD, decision log, data dictionary, meeting notes	The documents, where they can be found and versioned
Visio / Lucidchart / draw.io	As-is and to-be process maps, swimlanes	The process, drawn, including the handoffs
Figma / Balsamiq	Wireframes and clickable prototypes	Agreeing a layout before anything is built
Excel / SQL	Data profiling, reconciliation, the numbers behind a requirement	Checking that what the stakeholder believes is true

The last row is the one that distinguishes a data BA, and it is worth volunteering. A BA who can profile the source themselves — count the distinct values, find the 18 spellings of status, measure the match rate — writes requirements that survive contact with the data. One who cannot writes requirements that get rejected in development.

What not to do: claim a certification or a tool you have used once. The follow-up is always "what did you build in it", and a thin answer there is worse than not mentioning it.

Tier	What to say
Passes	Lists Jira, Confluence, Visio and Excel
Strong	Tool → artifact → outcome for two or three, including something they actually produced
Extra points	+ [+Business] mention the Jira key convention — the issue key in the branch name and pull request title, so a year later the code traces back to the request + [+Validate] volunteer the SQL/Excel profiling row, because writing requirements against data you have examined is the whole difference in a data BA + [+Clarify] ask what the team uses before describing a preference, since the answer is "whatever is already there"

Likely follow-ups: How would you structure a Confluence space so people find things? What do you put in a Jira ticket that developers actually need?

Red flag: a tool list with no artifact attached to any of them.

Learn it in: Chapter 26, §26.10 (Jira, issue keys, epics and stories); Chapter 26, §26.9 (a repository a stranger can run).

Q76B-077 · What is a wireframe, and when would you use one instead of a written requirement?

Level: Mid · **Roles:** BA

Remember it as: *A wireframe settles layout arguments in one meeting that prose cannot settle in five.*

Answer in one line: A wireframe is a deliberately rough sketch of a screen's layout and content — no colour, no final copy — used when the requirement is about **arrangement, hierarchy or flow**, because people cannot review a described layout but will immediately react to a drawn one.

	Use a wireframe	Use written requirements
The question is	"Where does this go, and what does the user see first?"	"What must be true of the number in that box?"
Riverstone example	The layout of the month-end dashboard	The revenue definition behind each tile

They are complements, not alternatives, and saying so is the answer. The wireframe shows the dashboard has four tiles; only the written requirement says which of Q76B-064's four revenue definitions each tile holds. A wireframe alone produces a pretty dashboard with undefined numbers.

Low fidelity on purpose. A rough sketch invites change; a polished mock-up invites approval. Early on you want the first, which is why keeping a wireframe deliberately ugly is a technique rather than laziness.

Tier	What to say
Passes	Defines it as a rough screen layout
Strong	Distinguishes layout questions from definitional ones and says the two artifacts are complements
Extra points	+ [+Business] low fidelity invites change and high fidelity invites sign-off, so the fidelity is a choice about what feedback you want + [+Edge cases] a clickable prototype is for testing a <i>flow</i> across screens, which a static wireframe cannot show + [+Validate] a wireframe with real example values on it catches definitional disagreements early, because a stakeholder who sees ₹44.26 crore will say whether that is the number they meant

Likely follow-ups: When would you move to a high-fidelity prototype? Who should be in the room when you review one?

Red flag: treating a wireframe as a design deliverable rather than a question-asking device.

Learn it in: Chapter 25, §25.6 (the four things a data team builds); Chapter 15, §15.11 (titles, labels, annotations, and clutter).

Q76B-078 · How would you estimate how long a requirement will take, when you are not the one building it?

Level: Mid · **Roles:** BA, AE

Remember it as: *You do not estimate the build. You make the requirement estimable, then give a range with its assumptions.*

Answer in one line: The estimate belongs to whoever does the work; the BA's job is to decompose the requirement until it can be estimated, surface the unknowns that make it unestimable, and convert the team's estimate into a range with stated assumptions rather than a date.

The four moves:

1. **Decompose** until each piece is comparable to something the team has done. 2. **Name the unknowns** that dominate — a source system nobody has queried, a rule nobody has written down. These need a **spike**, not a guess. 3. **Get the estimate from the builders.** A BA estimate presented as the team's is how commitments get made that nobody owns. 4. **Give a range, with the assumptions attached.** "Three to five weeks, assuming the finance extract is available and payment matching stays out of scope."

The honest line to hold. Estimates are not commitments, and the pressure to turn one into the other comes from above, not from the team. The BA's protection against it is the written assumption list: when an assumption breaks, the estimate visibly no longer applies, which is a conversation rather than a failure.

Tier	What to say
Passes	Says they would ask the developers
Strong	The four moves, with the unknowns-need-a-spike point and the range-with-assumptions output
Extra points	+ [+Validate] the assumption list is what makes a changed estimate defensible later, so it is the deliverable, not the number + [+Business] give a range and the confidence; a single date invites being held to the optimistic end of a distribution + [+Edge cases] a requirement nobody can estimate is usually one nobody understands, so the right output is a spike and a revised requirement + [+Signpost] t-shirt sizing (Q76B-057) is the right granularity when the question is a roadmap rather than a sprint

Likely follow-ups: What would you do if the team's estimate is rejected as too long? How would you handle an estimate that turns out to be badly wrong?

Red flag: giving a confident single-number estimate for work you are not doing.

Learn it in: Chapter 26, §26.10 (points, spikes, and the honest part); Chapter 25, §25.12.

Rapid-fire, 76B.12

Roles: BA for every row.

#	Question	One-line answer	Extra point	Level · learn it in
Q76B-079	What is a data dictionary, and who is it for?	A definition per field — meaning, type, allowed values, source, owner — written for whoever uses the data next, and it is the artifact that stops the same definitional question being asked every quarter	[+Signpost] it is where Q76B-064's four revenue definitions get settled once	Fresher · 25.7, 26.9
Q76B-080	What is an ERD, and does a BA need to read one?	An entity-relationship diagram showing tables and how they relate, including whether a relationship is one-to-many; a BA needs to read one to know whether a requirement is even possible against the model, and to spot the grain question before writing it	[+Signpost] one-to-many is Q76B-065's lines-versus-orders problem drawn as a diagram	Mid · 28.8, 28.5
Q76B-081	What is a CRUD matrix?	A grid of entities against roles or processes marking who can Create, Read, Update or Delete each one; it exposes the gaps — an entity nobody creates, or data everyone can update	[+Validate] it is the fastest way to find a missing requirement and an access-control problem at once	Mid · 25.5
Q76B-082	What is a RACI matrix, and what does it fix?	Responsible, Accountable, Consulted, Informed, per decision or deliverable; it fixes the two failures of having nobody accountable and having everybody consulted	[+Edge cases] exactly one Accountable per row, or it has not fixed anything	Fresher · 24.3
Q76B-083	What is a RAID log?	Risks, Assumptions, Issues and Dependencies, kept in one place and reviewed; the Assumptions column is the one that matters most, because an unreviewed assumption is how a project discovers a problem late	[+Signpost] this is where Q76B-043's labelled assumptions are meant to live	Mid · 24.6
Q76B-084	Five whys, or a fishbone diagram?	Five whys follows one causal chain and suits a single incident; a fishbone (cause-and-effect) spreads candidate causes across categories and suits a problem with several contributing factors	[+Trade-offs] five whys can stop at a convenient answer, so the discipline is asking who would disagree with each step	Mid · 24.1
Q76B-085	What is BABOK, and do you need a certification to be a BA?	BABOK is the IIBA's guide to business-analysis practice, and it is the vocabulary many interview questions are drawn from; a certification is not required for most roles and does not substitute for having shipped something, though it can help a CV past a screen	[+Clarify] requirements, fees and eligibility for these change, so check the IIBA's own current pages rather than trusting a summary	Fresher · 25.1
Q76B-086	What is a SWOT analysis, and	Strengths, Weaknesses, Opportunities, Threats, used to frame a strategic	[+Business] make each cell carry a "so what" and	Fresher · 24.6

#	Question	One-line answer	Extra point	Level · learn it in
	what is its failure mode?	option; its failure mode is a four-box list nobody acts on, because it produces categories rather than a decision	an owner, or it is a workshop artifact rather than an analysis	

76B.13 Domain scenarios

Interviewers increasingly open with a domain rather than a technique: "we're a logistics company — how would you approach this?" The question is testing whether you can find the few things that matter in an unfamiliar business, and the method is the same every time. **Ask what one row means, find the metric everyone argues about, and name the data that will turn out to be missing.**

Riverstone is a food wholesaler with four branches, so the logistics scenario below uses it directly. The others step outside the book's example, which is the point: the skill has to transfer.

Q76B-087 · A logistics company asks you to report on-time delivery. How do you specify it?

Level: Mid · **Roles:** BA, DA

Remember it as: "On time" is a comparison between two timestamps, and almost every argument about delivery performance is really an argument about which two.

Answer in one line: Specify it as an explicit comparison — which **promised** time, which **actual** time, measured at which event, with a stated tolerance — because each of those four choices moves the number, and the business will have an existing but unwritten convention for at least two of them.

The questions that have to be answered before the metric exists:

	The question	Why it moves the number
1	Promised against what ? The date quoted at order, the date confirmed after stock check, or the date re-promised after a delay	Re-promising is the big one: a supplier who re-promises can report 100% on-time while delivering everything late
2	Actual measured at which event ? Dispatch, arrival at the customer's gate, signed receipt, or the system being updated	Gate arrival and signature can differ by hours; the system update can differ by a day
3	What tolerance ? Same calendar day, within a time window, or to the minute	A same-day rule and a two-hour-window rule produce very different percentages from identical data
4	Which deliveries are in scope ? Cancelled, partial, customer-caused delays, returns	Excluding customer-caused delays is reasonable and is also the most common way the number is quietly improved

Question 1 is the one to raise first, because it is where the metric is most often gamed and it is almost never written down. A definition that does not fix the promise date can be satisfied by changing the promise.

On Riverstone's data this is not hypothetical. The order file has an `order_date` and an `entered_at_utc`, and Chapter 72B measured that on 1,604 of 25,969 rows the UTC date is not the Indian date. A same-day on-time rule evaluated in the wrong time zone misclassifies a predictable slice of evening deliveries, and the error runs one way.

The requirement, written:

> **BR-12.** Riverstone needs to measure delivery reliability by branch so that persistent lateness is visible before customers complain. > **FR-20.** A delivery is **on time** when the signed-receipt timestamp, converted to IST, falls on or before the **originally confirmed** delivery date. Re-promised dates do not replace the original. Cancelled orders are excluded; partial deliveries are on time only when the full ordered quantity is received. > **NFR-11.** The report states the definition and the exclusions on its face.

Tier	What to say
Passes	Asks for the definition of on-time and whether cancelled orders count
Strong	Names all four choices, identifies the promise-date question as the one most open to gaming, and writes the requirement explicitly
Extra points	+ [+Business] a metric that can be improved by re-promising will be, so fix the baseline in the definition + [+Validate] the time-zone point: a same-day rule needs a stated zone, and Chapter 72B measured 1,604 rows where it matters + [+Edge cases] partial deliveries need their own rule, or they get counted as on time by whoever benefits + [+Clarify] ask whether anyone already reports this number, because an existing unwritten definition is what you will be reconciled against

Likely follow-ups: How would you handle customer-caused delays? What would you do if receipt timestamps are missing for a third of deliveries?

Red flag: accepting "on-time delivery percentage" as a self-explanatory metric.

Learn it in: Chapter 23, §23.13 (defining a metric so two teams get the same number); Chapter 25, §25.5; Chapter 72B, §72B.4.

Q76B-088 · An e-commerce company wants to understand returns. Where do you start?

Level: Mid · **Roles:** BA, DA

Remember it as: *A return is not an event, it is a process with several states. Decide which state counts, and against which denominator.*

Answer in one line: Start with the grain and the denominator — is a return counted per item or per order, and is the rate measured against orders placed in a period or returns received in it — then establish the return **reasons**, because the reason data is what makes the analysis actionable and is almost always the weakest field in the system.

The specification questions:

1. **Per item or per order?** An order with four items and one returned is not a returned order. 2.

Which denominator? Returns received in March over orders placed in March mixes two populations, because a March return often belongs to a February order. The honest version is a **cohort**: of orders placed in February, what share has been returned so far. 3. **Which state counts** — return requested, authorised, received, refunded? They differ, and refunded is the one finance recognises. 4. **How long is the window?** Returns arrive over weeks, so a recent cohort always looks better than an old one. This is the single most common error in returns reporting. 5. **What are the reasons, and who enters them?** A free-text field, or a dropdown whose first option is "other", produces unusable reason data.

Point 4 deserves emphasis because it misleads in a specific, directional way: measured without a fixed window, the most recent month always shows the lowest return rate, and a chart of it shows a flattering downward trend that is entirely an artifact. The fix is to compare equal-maturity windows — 30 days after order, for every cohort.

Where the real finding usually is. Returns concentrate: a few products, a few sizes, a few sellers. The analysis that changes anything is not the overall rate but the breakdown, and that needs the reason field to be usable. **The most valuable thing a BA often delivers here is a requirement to fix the reason dropdown**, which is unglamorous and worth more than the dashboard.

Tier	What to say
Passes	Asks about the return rate definition and suggests breaking it down by product
Strong	Raises the cohort and maturity-window problem, and identifies reason-code quality as the constraint on the whole analysis
Extra points	+ [+Validate] equal-maturity cohorts, because otherwise recent periods always look better and the trend is an artifact + [+Business] the actionable output is usually a reason-code fix and a product-level breakdown, not an overall rate + [+Edge cases] fraud and serial returners need separate treatment and separate access rules + [+Clarify] ask whether the goal is to reduce returns or to reduce their cost; free returns may be a deliberate acquisition strategy

Likely follow-ups: How would you present a cohort view to a non-technical stakeholder? What if reason codes are 60% "other"?

Red flag: dividing this month's returns by this month's orders.

Learn it in: Chapter 23, §23.13; Chapter 23, §23.9 (customer metrics); Chapter 25, §25.3.

Q76B-089 · A bank asks you to analyse loan application drop-off. What do you need to be careful about?

Level: Senior · **Roles:** BA, DA

Remember it as: *A regulated domain changes what you may do with the data before it changes what you can learn from it. Ask about permissions first.*

Answer in one line: Be careful about three things in this order — what you are **permitted** to access and retain, what a "drop-off" actually means across a multi-session process, and whether any resulting

model or rule could produce **unfair or unexplainable outcomes** for applicants — and confirm the first with compliance before designing the analysis.

The analysis questions:

Grain	One applicant, one application, or one session? Applicants abandon and return days later, often on another device, so session-level drop-off vastly overstates abandonment
Definition of drop-off	Abandoned at a step, or never completed within a window? Without a window, every incomplete application is counted as a drop-off including ones still in progress
The funnel's real shape	Steps are rarely linear: applicants go back, re-upload documents, get referred for manual review. A linear funnel chart will be wrong
Decline against abandon	An application the bank rejected is not a drop-off, and conflating the two hides both problems

The part that makes this a senior question. In financial services the constraints are not optional and they come first:

- **Access and minimisation.** You may not need applicant identity at all to answer a funnel question, and the strongest position is to design the analysis so you never request the fields you do not need.
- **Retention and location.** Where the extract may live, how long, and who else can reach it are governance questions with real answers at the firm, not preferences.
- **Fairness and explainability.** If the output informs who gets contacted, re-offered or approved, then differential outcomes across groups are the risk, and "the model said so" is not an acceptable explanation to a customer or a regulator.

What to say about the rules themselves. Say that these constraints exist, that they are set by the firm's compliance function and the applicable regulator, and that you would confirm the specifics with them rather than asserting them. **Do not quote a particular regulation, retention period or threshold in an interview unless you genuinely know it applies** — a confident wrong citation in a regulated domain is far more damaging than saying "I would confirm that with compliance before writing the requirement."

Tier	What to say
Passes	Mentions that the data is sensitive and the analysis should be careful
Strong	Puts permissions first, then specifies grain, drop-off definition and the decline/abandon distinction, and raises fairness where the output drives a decision about a person
Extra points	+ [+Business] data minimisation is both a compliance position and a better analysis design: fewer fields, fewer approvals, faster delivery + [+Clarify] confirm the constraints with compliance rather than asserting them, and say so plainly + [+Edge cases] cross-device and cross-session returning applicants are the main measurement trap, and they make session-level drop-off unusable + [+Validate] an incomplete-but-active application is not a drop-off, so the definition needs a window

Likely follow-ups: How would you answer the funnel question without applicant identity? What would you check before a model's output was used to prioritise outreach?

Red flag: quoting a specific regulation or retention period you are not sure of; or treating fairness as a later concern.

Learn it in: Chapter 25, §25.5 (non-functional requirements, including access); Chapter 63, §63.9 (controls: approvals, segregation of duties, audit trails); Chapter 39, §39.5 (fairness and honesty about a model's limits).

Rapid-fire, 76B.13

Roles: BA and DA for every row.

#	Question	One-line answer	Extra point	Level · learn it in
Q76B-090	You are put on a domain you know nothing about. What do you do in week one?	Learn the process and the vocabulary before the data: follow one real transaction end to end, write down every term you do not know, and ask the three people who touch it what goes wrong most often	[+Business] Chapter 3's order 5001 is this exercise done once properly, and it is the single fastest way into any business	Fresher · 3.2, 25.4
Q76B-091	What does BFSI mean, and why does it get named separately in Indian job ads?	Banking, Financial Services and Insurance — grouped because they share regulated data, heavy audit requirements and similar core systems, so domain experience transfers across them and is valued accordingly	[+Clarify] it is a sector label rather than a skill; ask which of the three and which product line	Fresher · 25.1
Q76B-092	A warehouse client asks for "better inventory visibility". First question?	Visibility for what decision — knowing what to reorder, finding stock that is not moving, or explaining a stockout after the fact? Each needs a different measure and a different refresh frequency	[+Signpost] this is Q76B-070's pattern: the decision sets the requirement	Mid · 25.3, 50.1
Q76B-093	How would you define a "stockout" for a wholesaler like Riverstone?	Not simply zero stock: an order line that could not be filled from available stock when the customer wanted it, which requires demand data and not only inventory data — and the missing demand data is usually the finding	[+Business] stockouts measured only from inventory miss every sale that was never placed because the customer knew you were out	Senior · 23.8, 25.3
Q76B-094	A manufacturing client wants to reduce downtime. Where is the data problem?	Almost always in the event log's reason codes and in whether downtime start and end are captured automatically or typed by an operator afterwards, which decides whether any duration figure is trustworthy	[+Signpost] the same reason-code weakness as e-commerce returns in Q76B-088	Mid · 23.8, 14.5
Q76B-095	How do you handle a domain expert who contradicts the data?	Treat it as a genuine finding either way: the data may be wrong, or the expert may be describing the process as designed rather than as run — so trace one specific case together until you both see the same thing	[+Business] Chapter 14's branch league table is this situation resolved by looking at one row at a time	Mid · 14.2, 24.1, 25.11

Common mistakes

Mistake	Symptom	Fix
Documenting a vague ask as the requirement itself	Nobody can tell what "done" actually means	Ask for a specific example of the pain, restate as a testable requirement
Treating BRD/FRD/SRS or use case/user story as interchangeable	Wrong document for the audience, confusion downstream	Know the audience and detail level each one is for
A user story with no acceptance criteria	The story can't actually be tested or marked done	Always attach Given/When/Then criteria, including edge cases
A process map showing only the happy path	Exception handling gets built as an afterthought, or not at all	Map every branch at every decision point, including rejections
Assuming the BA's job ends once requirements are signed off	The delivered system quietly drifts from the original need	Stay involved through design, build, and UAT
Skipping the as-is map and jumping straight to to-be	A proposed improvement that doesn't actually address the real current process	Always document as-is first, however tempting it is to skip
Silently complying with (or flatly refusing) "can you just change the number"	Either damaged data integrity or an unresolved, escalating stakeholder relationship	Diagnose what's actually driving the request before responding

In the real world: the requirement that was actually three requirements

Farah, now a data analyst whose first project turned out to be requirements work, is asked to gather requirements for "a way to see which customers haven't ordered recently." She resists the urge to write that sentence down as the requirement and instead asks three different stakeholders, separately, what they'd actually do with that information.

Sales wants it to prioritize outreach calls, and only cares about customers likely to still be recoverable. Finance wants it for a different reason entirely: flagging dormant accounts that still have open balances for a bad-debt review, and closing their credit limits, with no interest in whether they're "recoverable." Customer success wants it for a third reason: proactively checking in with accounts before they become a support escalation, regardless of order recency alone.

What looked like one requirement was genuinely three, each needing a different definition of "haven't ordered recently," a different output format, and a different owner. Farah's actual deliverable ends up being three separate, precisely scoped requirements instead of one vague one, and the project's initial one-week estimate for "the recency report" becomes a more honest three-week estimate for three real features. Her manager's note: *"The original ask would have shipped something nobody could actually use well. Three real requirements, properly scoped, took longer to define but were faster to build right the first time."*

Project

Goal: run this chapter's full method on a real process, start to finish.

Tools you'll need

No software specific to this chapter. A whiteboard or a simple diagramming tool (Lucidchart, draw.io, or even PowerPoint) for process maps; a shared document for BRDs, FRDs, and user stories; Chapter 11's spreadsheet skills (the same ones Chapter 25, section 25.7 uses for a traceability matrix) for a requirements traceability matrix in practice.

1. Pick a real, recurring process you're familiar with (at work or elsewhere), and interview at least two people involved in different steps of it. 2. Map it as-is, including at least one exception path, using a swimlane format. 3. Write one user story with full Given/When/Then acceptance criteria for a specific improvement to that process. 4. Draft the requirement, in one sentence, that a gap analysis of the same process would produce. 5. Write the "can you just change the number" response you'd actually give, for a real or hypothetical version of that request in your own context.

Key terms

elicitation · business requirement · functional requirement · non-functional requirement · business rule · requirements traceability matrix · stakeholder register · BRD · FRD · SRS · user story · use case · INVEST · Given/When/Then (Gherkin) · reconciliation criterion · Definition of Done · spike · swimlane diagram · handoff · BPMN · decision point (gateway) · as-is vs. to-be · gap analysis · root cause · SDLC · product backlog vs. sprint backlog · Product Owner · change request · UAT · UAT sign-off

Final-week revision list

Q76B-001, Q76B-002, Q76B-007, Q76B-008, Q76B-013, Q76B-018, Q76B-023, Q76B-028, Q76B-029, Q76B-033, Q76B-038.

Where this leads

- **Chapter 69, The Extra-Points Method**, is the rubric and move set every answer above is written against, especially Move 1 (clarify) throughout sections 76B.2 and 76B.8.
- **Chapter 75, Product Sense, Metrics, Case Studies & Guesstimates**, already covers case-structure and stakeholder reasoning from the diagnostic-case angle; this chapter covers the same discipline from the documentation and process side.
- **Chapters 24 and 25** teach every technique this bank tests, and Chapter 26, section 26.10 the Agile vocabulary; each question's "Learn it in" names the section.
- **Chapter 78, Automation & Integration Question Bank**, takes automation from the requirement (Q76B-038) to the build: choosing the tool, retries, idempotency, and alerting.